Project Report

SignBridge

Bidirectional Indian Sign Language and English Communication with a 3D Avatar

Darshan BG

3 November 2026

A research prototype. Every result is on the authors' predefined split of INCLUDE, signer independence not verified. No avatar clip has been reviewed by a signer. INCLUDE-derived material is shared for non-commercial use only under CC BY-NC-SA 4.0.

Abstract

SignBridge is a research prototype that goes in both directions between Indian Sign Language (ISL) and English in a web browser. For sign to English, the webcam is read in the browser with MediaPipe Holistic, only hand and body landmarks (never video) are sent to a FastAPI server, and a one-dimensional convolutional network trained on the INCLUDE dataset (262 classes) recognizes one sign at a time, with a continuous mode that cuts a landmark stream into signs. For English to sign, a spaCy rule layer converts a sentence into ISL gloss and a 3D avatar plays one retargeted clip per gloss. Gloss is the only representation the two directions share.

On the authors' predefined split of INCLUDE (signer independence not verified) the live network reaches a top-1 accuracy of 0.9130 and, at the confidence threshold of 0.7, answers 0.8676 of the test samples with 0.9548 accuracy. The report then measures what a deployment changes. On 100 test videos at that threshold the same model answers 0.82 correctly from stored features, 0.78 from browser landmarks of 16:9 frames, 0.65 from 4:3 frames and 0.07 from 4:3 frames when the picture aspect is ignored, with 29 percent confident wrong answers. A targeted augmentation (vertical scale, landmark noise, hand dropout) is evaluated on three architectures; the three-seed numbers are **PENDING**. The report documents the implementation, 367 backend and 137 frontend automated tests, the limits of the work (no face features, no signer-reviewed clips, an unresolved dataset licence) and the steps to run and deploy the system.

Table of Contents

1 Introduction 1

1.1 Problem statement 1

1.2 Background on Indian Sign Language 1

1.3 Aim 1

1.4 Objectives 1

1.5 Scope and what is not claimed 2

1.6 Contributions 2

1.7 Organization of this report 2

2 Literature Survey 3

2.1 Sign language datasets and recognition 3

2.2 Landmark extraction 3

2.3 Sequence models 3

2.4 Dataset shift, robustness and augmentation 3

2.5 Abstention and calibration 4

2.6 Statistics used 4

2.7 Sign language technology with Deaf communities 4

2.8 Gap addressed by this project 4

3 Existing System and Proposed System 5

3.1 Existing approaches 5

3.2 Proposed system 5

3.3 Advantages and honest limits 6

4 Requirements 7

4.1 Hardware requirements 7

4.2 Software requirements 7

4.3 Functional requirements 7

4.4 Non-functional requirements 8

5 System Design 9

5.1 Architecture 9

5.2 Sign to English pipeline 10

5.3 English to Sign pipeline 10

5.4 Gloss as the shared representation 11

5.5 Data flow 12

5.6 Module design 12

5.7 Storage 13

5.8 Application programming interface 13
6 Dataset 14
6.1 The INCLUDE dataset 14
6.2 Processing and audit 14
6.3 Split 14
6.4 Preprocessing and normalization 15
6.5 Licence 16
6.6 What the dataset cannot show 16
7 Implementation 17
7.1 Recognition models 17
7.2 Training pipeline, augmentation and checkpoints 17
7.3 Live recognition and the threshold 17
7.4 Continuous signing 18
7.5 Text to gloss 18
7.6 Gloss to English 18
7.7 Retargeting landmarks to avatar clips 18
7.8 The avatar 19
7.9 Frontend and user interface 19
7.10 Backend 23
7.11 Recorder 23
7.12 Deployment 24
8 Testing 25
8.1 Test strategy 25
8.2 Automated tests 25
8.3 Integration tests 26
8.4 Browser-path benchmark 26
8.5 End-to-end tests with a fake camera 26
8.6 Manual verification 26
8.7 What testing does not show 27
9 Results 28
9.1 Model comparison 28
9.2 Threshold trade-off 29
9.3 Deployment-gap results 30
9.3.1 Browser path against the offline path 30
9.3.2 Vertical scale, noise and hand dropout 31
9.3.3 Signer size in the picture 32

9.4 Augmentation 33
9.5 Segmentation 34
9.6 Comparison with published INCLUDE results 35
10 Limitations and Future Work 36
10.1 Limitations 36
10.2 Problems found in commits made outside the project sessions 37
10.3 Test-split disclosure 37
10.4 Future work 38
11 Conclusion 39
References 40
Appendix A: Setup and Run Steps 42
A.1 Setup 42
A.2 Files that are not in the repository 42
A.3 Run 42
A.4 Using the application 42
A.5 Tests 43
A.6 Training and evaluation 43
A.7 Deployment 43
A.8 Rebuilding this report 43
Appendix B: API Reference 44

List of Figures

Figure 1**.** System architecture of SignBridge. Solid arrows are run-time data flow; dashed arrows show which files each part reads. 9
Figure 2**.** Sign to English pipeline. Continuous mode adds the segmenter in front of the same classifier. 10
Figure 3**.** English to Sign pipeline. A gloss without a clip is skipped and reported. 11
Figure 4**.** Gloss is the only thing the two directions share. 11
Figure 5**.** One INCLUDE test sample of the sign Hello drawn from its 225 stored features at eight of its 32 resampled frames. Only landmarks are drawn; the circle marks the nose position and is not a face. 15
Figure 6**.** The offline data and training pipeline. 16
Figure 7**.** The avatar across the sentence "Hello. Thank you. Good morning. Doctor." at half speed, eight frames from the production build. The avatar is a mannequin; its face is not animated, and no clip is reviewed by a signer. 19
Figure 8**.** Conversation tab: a signed message, a typed message that the avatar is signing, the camera with its overlay and the system status. The camera feed is an INCLUDE test video with the head masked. 20
Figure 9**.** Sign to English tab with the fake camera: landmark overlay, the dashed framing guide, the buffer counters and an isolated recognition result. 21
Figure 10**.** English to Sign tab: the gloss of "Hello, thank you", the avatar in the middle of the sign HELLO and the playback controls. The page continues with the vocabulary list. 22
Figure 11**.** Record sign tab (single sign mode) with the camera running. Nothing is recorded until the user presses the record button. 22
Figure 12**.** Landmark check tab with the camera running. It records a short video and the browser landmarks locally for the offline comparison. 23
Figure 13**.** Test results of the three baseline models. 28
Figure 14**.** The most frequent confusions of the 1D CNN on the test split. 29
Figure 15**.** Confusion matrix restricted to the 14 classes with the most errors (1D CNN, test split). .. 29
Figure 16**.** Accuracy against coverage (left) and coverage against the threshold (right) for the three baseline models on the test split. The mark 0.7 is the live threshold. 30
Figure 17**.** The 1D CNN on 100 test videos at threshold 0.7: correct-and-accepted share with 95 percent Wilson intervals and wrong-and-accepted share, offline and in the browser path. 31
Figure 18**.** The 1D CNN against a vertical scale error applied to browser landmarks of 100 validation videos. The shaded band is the range of the augmentation. 32
Figure 19**.** The 1D CNN against the median shoulder width, validation videos shrunk toward the centre. The dotted line is the warning threshold of the application. 33
Figure 20**.** Segmentation results on validation and test streams for three amounts of rest between signs. 35

List of Tables

Table 1. Comparison of existing approaches and the proposed system. 5
Table 2. Hardware used and hardware required. 7
Table 3. Software and library versions. 7
Table 4. WebSocket messages of the live recognizer. 12
Table 5. Backend modules. 12
Table 6. Frontend modules. 12
Table 7. Files used as storage. 13
Table 8. Audit of the processed INCLUDE data. 14
Table 9. Samples per split. 14
Table 10. The three recognition models. 17
Table 11. Automated test suites. 25
Table 12. The largest backend test files. 25
Table 13. End-to-end test with a fake camera before and after the aspect rescale (isolated and continuous modes, one run each). 26
Table 14. Test results of the baseline models (seed 0). 28
Table 15. Coverage and accuracy on answered samples against the confidence threshold (test split, baseline models, seed 0). 30
Table 16. The 1D CNN on 100 test videos at threshold 0.7. 30
Table 17. The 1D CNN on browser 16:9 landmarks of 100 validation videos with added noise or removed hands, threshold 0.7. 32
Table 18. The 1D CNN by shoulder width, validation videos shrunk toward the centre (Python MediaPipe, threshold 0.7). 32
Table 19. Test results of the baseline and augmented models, single seed (seed 0). Differences between single runs are not separated from training variance. 33
Table 20. Three-seed summary, mean and spread over seeds 0, 1 and 2, accuracy at the validation-chosen threshold unless stated (authors' predefined split, signer independence not verified). Test videos are evaluated once per trained model. 34
Table 21. Segmentation results on streams of real INCLUDE samples (real signs split correctly, recognized correctly, accepted but wrong, UNRECOGNIZED). 34
Table 22. Files that are not in the repository and what happens without them. 42
Table 23. REST and WebSocket endpoints. 44
Table 24. Configuration through environment variables (prefix SIGNBRIDGE_). 44

1 Introduction

1.1 Problem statement

Deaf and hard-of-hearing people in India use Indian Sign Language (ISL), a visual language with its own vocabulary and grammar that is distinct from English and from American Sign Language. A hearing person who does not know ISL, and a deaf person who cannot rely on written English, often have no shared language. Tools that translate in both directions would help, but the technical problem is hard: signs must be recognized from video, English must be mapped to a sign-like structure, and the result must be shown as signing that a viewer can follow. Resources for ISL are far smaller than for languages with long research histories, and a system built on them must be honest about what it can and cannot do.

SignBridge is an end-to-end prototype of this problem. It recognizes isolated and continuous signs from a webcam, builds an English sentence from them, converts typed English to ISL gloss, and signs the gloss with a 3D avatar. It is an academic prototype, not a complete ISL translator, and every claim in this report is tied to a file in the repository or to a measured result.

1.2 Background on Indian Sign Language

ISL is produced with the hands, the arms and the upper body, together with facial expression and head movement that carry grammatical meaning such as questions and negation. Written descriptions of signs often use a gloss: an upper-case word label for each sign, in sign order, with no grammar words that the language does not sign. A gloss sequence is not a translation and not a writing system, but it is a convenient shared representation between text and signing, and it is the central representation of this project.

Machine recognition of signs usually starts from video. A pose estimator can reduce each frame to a few dozen landmarks for the body and hands, so that a classifier works on numbers instead of pixels. This makes models small and reduces the amount of personal appearance information that is processed, which is useful for privacy. It also discards what the estimator does not capture, such as most of the face, and it makes the system depend on the estimator behaving the same way on the training videos and on a live camera.

1.3 Aim

The aim is to build and honestly evaluate an end-to-end bidirectional ISL and English prototype that runs in a web browser, that keeps the user's video on their own machine, and that reports what it could not do instead of guessing.

1.4 Objectives

- Extract body and hand landmarks in the browser and send only landmarks to the server.
- Process the INCLUDE dataset completely, audit it, and train and compare three sequence classifiers on its authors' predefined split.
- Serve the best classifier live, with a confidence threshold chosen on validation data and an explicit "unrecognized" answer.
- Recognize signs in a continuous stream by segmenting it from hand motion and rest.
- Convert English text to ISL gloss with an editable rule layer and convert gloss back to English heuristically.

- Retarget landmark samples to avatar clips and play them on a 3D avatar with smooth transitions, without claiming that any clip is correct ISL.
- Measure how much accuracy is lost between the dataset test split and the browser path, and test a targeted augmentation against that loss.
- Provide a recorder so that new signs and signer-labelled takes can be collected, and prepare a deployment on Vercel and Hugging Face Spaces.

1.5 Scope and what is not claimed

In scope are isolated sign recognition for the 262 INCLUDE classes, continuous recognition with synthetic pauses tested on real INCLUDE samples, text to gloss for simple single-clause sentences, and avatar playback of unreviewed clips. Not claimed are complete natural-language translation, ISL grammar including facial markers, fingerspelling, any estimate of accuracy for a new signer on a real webcam, and the correctness of any avatar clip. Results that depend on the number of seeds or on data that has not been collected are marked as pending in red.

1.6 Contributions

- A working bidirectional prototype with a shared gloss representation, a browser-side landmark pipeline and a production build.
- A complete processing and audit of INCLUDE (4,284 samples, 262 classes) and a comparison of a 1D CNN, a BiGRU and a Transformer under identical settings.
- A browser-path benchmark that exposes a deployment gap, including a picture-aspect error that normalization by shoulder width does not remove, and the changes made in response (aspect rescale, camera request, framing guide, size warning).
- A targeted augmentation with an exact vertical-scale transform, evaluated against the deployment gap.
- A documented list of every use of the test split and of every limitation.

1.7 Organization of this report

Chapter 2 surveys related work and Chapter 3 compares existing and proposed systems. Chapter 4 lists the requirements and Chapter 5 describes the design. Chapter 6 describes the dataset and its preprocessing, and Chapter 7 the implementation. Chapter 8 describes the testing and Chapter 9 the results. Chapter 10 states the limitations and future work and Chapter 11 concludes. The appendices give the setup and run steps and the API reference.

2 Literature Survey

This chapter surveys the work that this project builds on. It uses only the references of the accompanying paper; each of them is still marked for checking in `paper/refs.bib` before submission, and this report inherits that status.

2.1 Sign language datasets and recognition

Word-level datasets turned isolated sign classification into a standard benchmark. WLASL provides a large American Sign Language collection [1], and INCLUDE provides a word-level collection for Indian Sign Language that is the basis of this project [2]. Surveys describe the field of sign language recognition and its deep learning methods [3]. Skeleton-based and landmark-based methods are among the strongest and cheapest systems [4]. OpenHands collects pose-based models and pretrained encoders across several sign languages, including INCLUDE [5]. The figures it reports for INCLUDE are not repeated here because they have not been checked against the published version; Section 9.6 only compares the order of magnitude.

Two more recent resources show where the field is going. ASL Citizen is a community-sourced isolated sign dataset whose recorders are identified, which allows signer-disjoint splits [6]. The 2023 Google Isolated Sign Language Recognition competition on Kaggle made landmark-based isolated sign recognition an open benchmark [7]. INCLUDE has no signer identifiers, which is the root of the main limitation of this project.

All of these works evaluate on held-out splits of the same collection. This project additionally evaluates the path that a user would run.

2.2 Landmark extraction

MediaPipe provides body pose and hand landmarks in real time on commodity hardware and in the browser [8], [9], [10]. SignBridge uses MediaPipe Holistic unchanged on both sides of its comparison, in Python for the dataset and as a WebAssembly build in the browser, and measures how much of the observed gap is the extractor and how much is the input.

2.3 Sequence models

Three families of sequence classifiers are compared in this project: a one-dimensional temporal convolutional network, a bidirectional gated recurrent unit network [11], [12] and a Transformer encoder [13]. All three read a fixed-length sequence of 225 normalized landmark values per frame.

2.4 Dataset shift, robustness and augmentation

A mismatch between training and use is a long-studied problem [14], [15]. Modern benchmarks cover distribution shift in the wild [16] and corruption robustness [17], and equally accurate models can behave differently under shift [18]. The shifts studied here are specific and physical: picture aspect, signer size in the picture and detector dropout. This allows them to be measured one at a time and an augmentation to be chosen from the measurements instead of from convention [19].

2.5 Abstention and calibration

A reject option trades coverage for accuracy [20] and depends on confidence being meaningful, which deep networks do not guarantee [21]. The live recognizer answers UNRECOGNIZED below a threshold chosen on validation data, and this report shows the coverage and accuracy trade-off per condition.

2.6 Statistics used

Proportions are reported with Wilson score intervals [22]. Differences between paired runs on the same videos use the exact McNemar test [23] and paired bootstrap intervals [24].

2.7 Sign language technology with Deaf communities

Technology that claims to translate sign language has to be developed and evaluated with signers and their communities [25]. This project could not do that within its scope, and Chapter 10 states what that implies for every result.

2.8 Gap addressed by this project

The surveyed work trains and evaluates recognizers on dataset splits. What it does not usually report is the accuracy of the complete path that a user runs in a browser, the sensitivity of that path to the camera and framing, and the product behavior that follows from such measurements. SignBridge adds this measurement, the targeted response to it, and a bidirectional prototype around it.

3 Existing System and Proposed System

3.1 Existing approaches

Communication between a deaf signer and a hearing non-signer is supported today in four broad ways, none of which is a bidirectional, privacy-preserving, in-browser ISL tool built on an open, audited dataset.

- **Human interpreters** are accurate and understand grammar and context, but they are scarce, must be arranged in advance and are not available for everyday exchanges.
- **Written text and typing** work only if both people read and write a shared written language fluently.
- **Video recordings and dictionaries of signs** show one sign at a time. They answer "what does this sign look like" but not "what did that person just sign".
- **Research recognizers** classify isolated signs from video or landmarks and report accuracy on held-out splits of their own dataset [1], [2], [4], [5]. They typically go one way (sign to text), do not include a generator, and do not report the accuracy of the path that a user would run in a browser.

The limitations of these approaches that SignBridge addresses are the one-way direction, the missing signing output, the unreported deployment gap and the lack of an explicit "unrecognized" answer.

3.2 Proposed system

SignBridge is a web application with a Python backend. It has two directions that meet at the gloss representation (Chapter 5).

- **Sign to English.** The browser extracts landmarks and sends only landmarks. The server recognizes isolated signs, or cuts a stream into signs and recognizes each, and builds a gloss sequence and an English sentence. Answers below the confidence threshold are reported as unrecognized.
- **English to sign.** The server converts English to gloss with an editable rule layer. The browser plays one clip per gloss on a 3D avatar and lists the words that have no clip.
- **Conversation view.** One screen with a chat log for both directions.
- **Recorder.** A tool to record new signs or signer-labelled takes as landmarks only.
- **Measurement tools.** A landmark check that compares the browser landmarks with Python MediaPipe on the same recording, and a benchmark of the browser path.

Table 1 compares the existing approaches with the proposed system on the properties that matter for this project.

Table 1. Comparison of existing approaches and the proposed system.

Property	Interpreter	Typing	Sign dictionary	Research recognizers	SignBridge
Sign to text	yes	no	no	yes, isolated	yes, isolated and continuous
Text to sign	yes	no	one sign at a time	no	yes, with an avatar
Works without a person	no	yes	yes	yes	yes
Video stays on the user's device	not applicable	yes	yes	not usually stated	yes, only landmarks leave the browser
States when it	not applicable	not applicable	not applicable	not usually	yes, UNRECOGNIZED

Property	Interpreter	Typing	Sign dictionary	Research recognizers	SignBridge
cannot answer					and missing clips are listed
Deployment accuracy reported	not applicable	not applicable	not applicable	not usually	yes, browser path benchmark
Reviewed by signers	not applicable	not applicable	varies	no	no, stated as a limitation

3.3 Advantages and honest limits

The main advantages are the shared gloss, the privacy design, the measured and reported deployment gap and the refusal to invent output. The limits are that the system covers 262 recognizable signs and some grammar words that cannot be recognized, that no clip is signer-reviewed, that facial expression is not used, and that a new signer on a real webcam is unmeasured (Chapter 10).

4 Requirements

4.1 Hardware requirements

The system was developed and measured on one machine, so the numbers in this report describe that machine.

Table 2. Hardware used and hardware required.

Item	Development machine	Minimum for running the prototype
Processor	Intel Core i5-13450HX, 16 logical processors	A recent multi-core CPU; no GPU is required
Memory	Not a limiting factor	Enough for a Python process with PyTorch and spaCy, and a browser
Camera	640x480 and 1280x720 sources, real and fake	A webcam; 1280x720 is requested and any size is accepted
Graphics	Software WebGL in headless tests, a GPU in normal use	WebGL 1 or 2 for the 3D avatar; a built-in figure is used if the avatar file is missing
Disk	Several gigabytes for the INCLUDE archives and features	Small for running (model about 1.4 MB); large only for training
Operating system	Windows 11	Windows for the provided scripts; the code is portable

Model inference takes about 0.4 ms per sign on the development CPU (median of 200 runs), so network and landmark extraction dominate the delay.

4.2 Software requirements

Table 3. Software and library versions.

Layer	Software	Version or note
Runtime	Python	3.11
Runtime	Node.js	20 or newer (22 in the container build)
Version control	Git	any recent version
Backend	FastAPI, Uvicorn, Pydantic, pydantic-settings, Starlette	0.162.0, 0.32.1, 2.10.3, 2.6.1, 1.10.0
Backend	NumPy, PyTorch (CPU build)	2.1.3, 2.13.0
Backend	spaCy with en_core_web_sm	3.8.7 with 3.8.0
Data processing	MediaPipe (Python), OpenCV	0.10.14, 4.10.0.84
Frontend	Next.js, React, TypeScript, Tailwind CSS	16, 19, TypeScript, Tailwind
Frontend	React Three Fiber, drei, three	9.8, 10.7, 0.169
Frontend	MediaPipe Holistic (WebAssembly), self-hosted	0.5.1675471629
Testing	pytest, Vitest, ESLint, TypeScript compiler	as installed by the setup script
Browser	A current Chrome or Edge	camera and WebGL permissions
Optional	Docker Desktop	to build the Hugging Face Space image

4.3 Functional requirements

- **FR1.** Read the webcam in the browser and extract pose and hand landmarks without sending video to any server.
- **FR2.** Recognize one isolated ISL sign from a buffer of landmarks and return the label, the confidence and the alternatives.
- **FR3.** Return UNRECOGNIZED instead of a label when the confidence is below the threshold chosen on validation data.

- **FR4.** Recognize signs in a continuous stream by finding where each sign starts and ends, and build a gloss sequence and an English sentence with undo and clear.
- **FR5.** Convert typed English to an ISL gloss sequence with non-manual marker tags, and list the words that cannot be signed.
- **FR6.** Play one clip per gloss on a 3D avatar with smooth transitions, speed control, pause and replay.
- **FR7.** Offer a conversation view with a chat log for both directions.
- **FR8.** Record new signs and signer-labelled takes as landmarks only, with a quality check and clear reasons for rejection.
- **FR9.** Compare the browser landmarks with Python MediaPipe landmarks on the same recording.
- **FR10.** Show the state of every component (backend, camera, landmark extraction, model, clips) from real checks.
- **FR11.** Guide the user to a good camera framing and warn when the person is too small in the picture.
- **FR12.** Allow a deployment in which recording is disabled and the notices about data, licence and review are always shown.

4.4 Non-functional requirements

- **Privacy.** Video never leaves the browser. Only landmark arrays are sent, and the public deployment stores nothing.
- **Honesty.** The system never invents a prediction, a metric or a clip. Missing models, clips and words are reported.
- **Performance.** Model inference in milliseconds; the production build loads the page in 0.25 s against 0.9 s in development mode, measured on the development machine.
- **Reliability.** Every stage reports what it could not do (no model: 503; unusable landmarks: 422; unsupported words; missing clips; unreachable backend: a banner).
- **Reproducibility.** The same preprocessing function serves training and inference; training is seeded and can resume from checkpoints; results are produced by scripts.
- **Usability and accessibility.** Keyboard shortcuts for the main actions, touch targets of at least 44 pixels and a layout that works down to a 375 pixel wide screen.
- **Portability and deployment.** A production build served locally with one script, and prepared (not performed) deployments on Vercel and Hugging Face Spaces.
- **Licence compliance.** INCLUDE-derived material is handled under the stricter CC BY-NC-SA 4.0 reading and kept out of the public repository.
- **Testability.** Automated tests for the backend and the frontend, and a benchmark of the browser path.

5 System Design

5.1 Architecture

SignBridge has three parts: a browser application, a Python backend and a set of files on disk (Figure 1). The browser extracts landmarks, draws the avatar and holds the user interface. The backend normalizes landmarks, classifies them, segments streams, converts between English and gloss, and serves the clip list. Nothing is stored on the server at run time except takes made with the recorder.

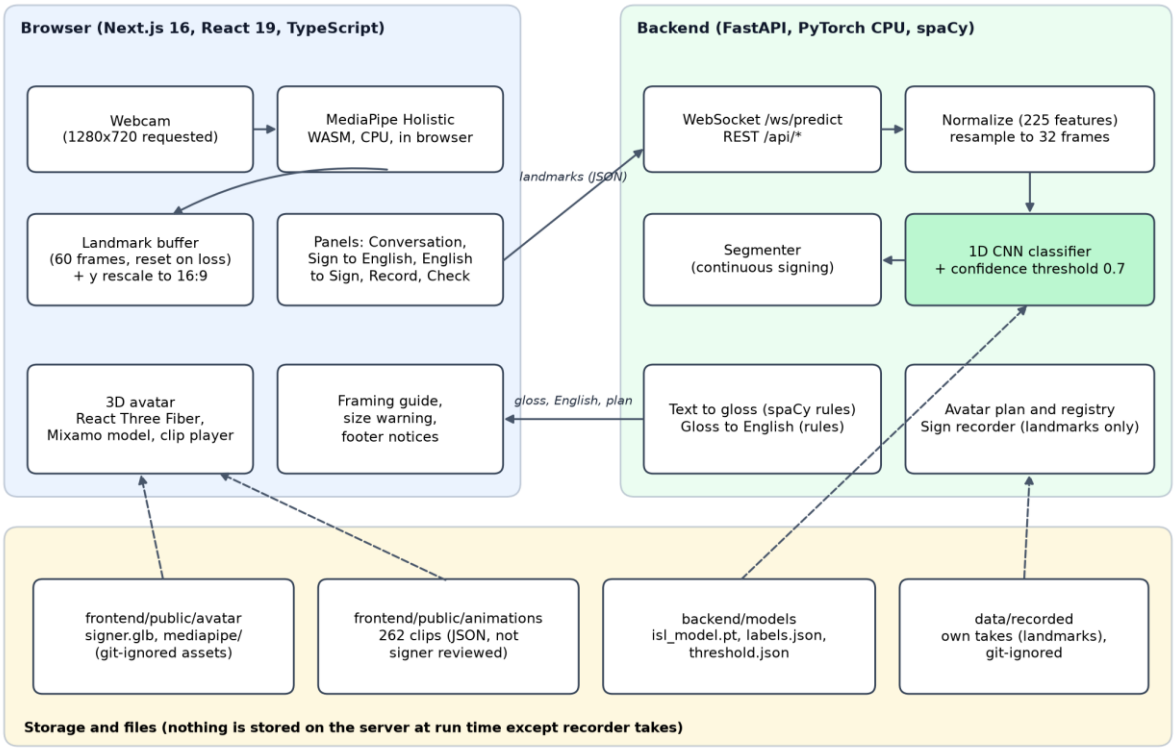


Figure 1. System architecture of SignBridge. Solid arrows are run-time data flow; dashed arrows show which files each part reads.

The main design decisions are the following.

- **Landmarks are extracted in the browser.** Video never leaves the machine. The backend receives arrays of 33 pose and 2 x 21 hand landmarks. Python MediaPipe is used only offline for the dataset, with the same normalization code, so training and inference features match.
- **One preprocessing path.** The normalization, the gap filling and the resampling are the same functions for training, for the REST endpoints and for the WebSocket.
- **Gloss is the shared representation.** The two directions do not share a model. They share only the gloss vocabulary and its dictionary.
- **Missing parts are reported, not hidden.** Without a model the live endpoint answers with an explicit error and closes; without a clip the word is listed as unsigned; without the avatar file a built-in figure is used.
- **Files instead of a database.** The model, the clips, the registries and the recorded takes are files; no database is needed for the prototype.

5.2 Sign to English pipeline

Figure 2 shows the path from the webcam to an English sentence. In isolated mode the browser sends the newest buffered frames when the user asks for a recognition. In continuous mode it streams chunks of frames every 250 ms and the server segments them before classifying.

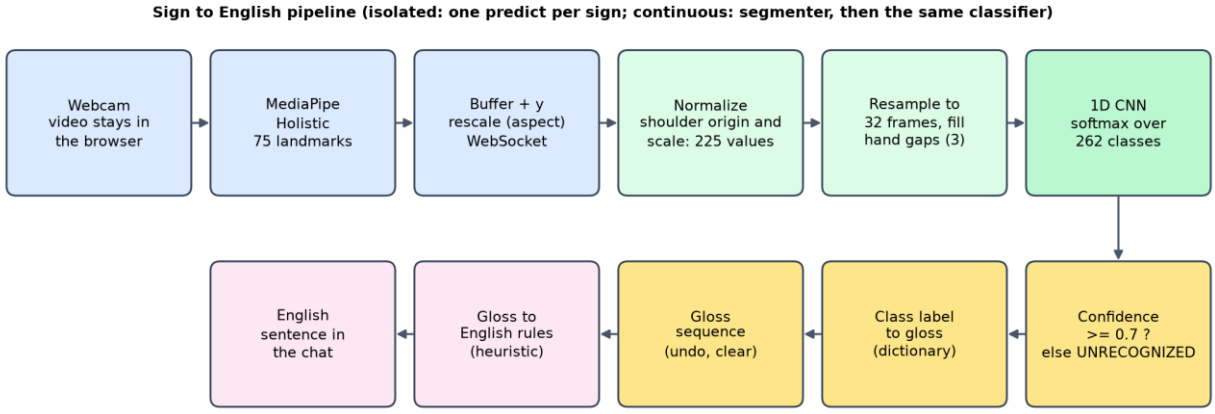


Figure 2. Sign to English pipeline. Continuous mode adds the segmenter in front of the same classifier.

The steps are as follows.

1. MediaPipe Holistic returns 33 pose landmarks and 21 landmarks per hand for each video frame.
2. The browser keeps a buffer of the newest 60 frames and drops it after 15 frames without a body pose, so an old partial sign is never mixed with a new one. It rescales the vertical coordinate by the picture aspect (Section 7.9) and sends the frames.
3. The server normalizes each frame: the shoulder midpoint becomes the origin and the shoulder distance the unit, giving 225 values per frame. A missing hand is a block of zeros. Frames without a usable pose are dropped.
4. The sequence is resampled to 32 frames after short hand gaps (three frames or fewer) have been interpolated.
5. The 1D CNN returns a probability for each of the 262 classes. If the top probability is at least the threshold (0.7), the label is accepted; otherwise the answer is UNRECOGNIZED.
6. An accepted label is mapped to its gloss, added to the sequence, and the gloss sequence is turned into an English sentence by rules.

5.3 English to Sign pipeline

Figure 3 shows the path from typed text to the avatar. The server converts the text to gloss, and the browser plans the playback from the clips that exist.

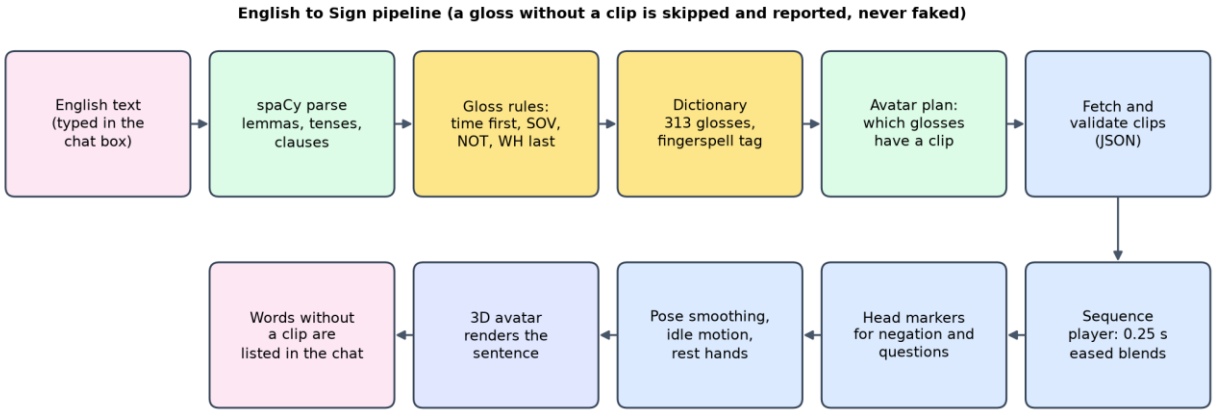


Figure 3. English to Sign pipeline. A gloss without a clip is skipped and reported.

1. spaCy parses the sentence into tokens, lemmas, tenses and clauses.
2. Rules order the gloss: time signs first, subject and objects in English order, verbs after their objects, negation after the verbs, and question words last. Articles, the copula, auxiliaries and prepositions are dropped and reported with a reason. Unknown words become fingerspelling markers.
3. The dictionary (313 glosses) maps each gloss to a sign entry, and the avatar plan marks which glosses have a clip.
4. The browser fetches and validates the clips and the sequence player blends them with eased transitions while the avatar applies smoothing and a small idle motion.
5. Non-manual markers (negation, yes-no question, WH question) add a rule-based head movement; the words without a clip are listed in the chat.

5.4 Gloss as the shared representation

Figure 4 shows why the two directions can be built and tested separately. Text and signs never meet directly. Text goes to gloss and back by rules; signs go to gloss through the class label of the recognizer and from gloss to clips through the registry. Adding a sign means adding a dictionary entry, a class or a clip, without touching the other direction.

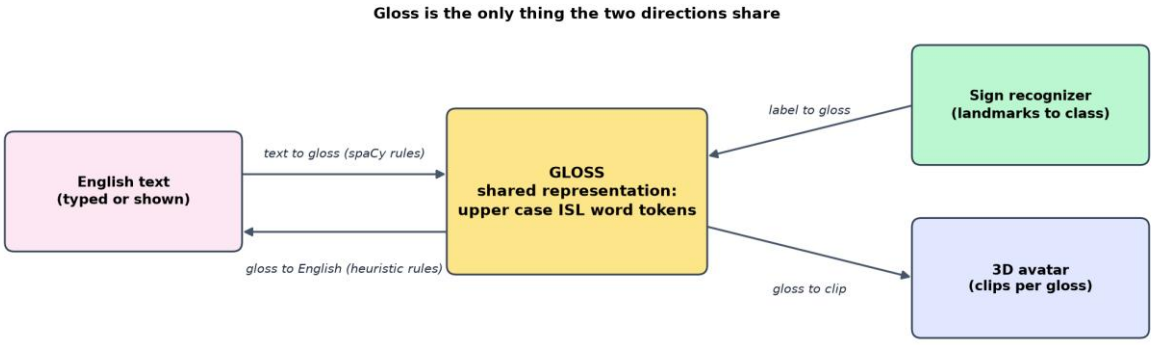


Figure 4. Gloss is the only thing the two directions share.

5.5 Data flow

The live channel is a WebSocket at /ws/predict. The client sends JSON text messages and the server answers with JSON events.

Table 4. WebSocket messages of the live recognizer.

Direction	Message	Meaning
server to client	ready	model loaded: number of classes, architecture, threshold and where the threshold came from
client to server	predict (frames, commit)	recognize one isolated sign from the frames and optionally add it to the sentence
server to client	prediction	accepted or not, label, gloss, confidence, alternatives, number of frames used, gloss sequence and English sentence
client to server	frames	continuous mode: a chunk of 1 to 300 frames
server to client	sign	one finished sign with its segment and the same fields as a prediction
server to client	stream	status of the continuous stream: active, frames seen, sequence, English
client to server	undo, reset, flush	remove the last sign, clear everything, close an open sign
server to client	error	without a model it sends model_not_trained and closes; other errors name the problem

The REST endpoints (Appendix B) cover health and status checks, landmark validation, one-shot recognition, translation in both directions, the avatar plan and registry, and the recorder.

5.6 Module design

Table 5. Backend modules.

Package or file	Responsibility
app/main.py, app/core	application, CORS, body size limit, settings read from SIGNBRIDGE_ environment variables
app/api	routes: REST, WebSocket, recorder
app/recognition	landmark validation and normalization, preprocessing, model definitions, artifact loading, inference, model status, segmenter
app/services	live session handling, recognition orchestration, avatar plan
app/avatar	clip format and validation, clip registry, sign registry with provenance
app/recording.py	quality check, takes on disk, clip building from a take
nlp	spaCy text to gloss, gloss to English, dictionary loader, fingerspelling
retarget	landmarks to avatar bones, clip builder, vocabulary and promotion tools
training	dataset tools, audit, training, comparison, calibration, augmentation, evaluation scripts

Table 6. Frontend modules.

Folder	Responsibility
src/app	the single page with five tabs and the footer
src/components	panels (conversation, sign to English, English to sign, record, landmark check), the camera hooks, the framing guide and the status panel
src/lib	landmark conversion and buffers, MediaPipe loader, overlay drawing, camera request, WebSocket client, recorder, shortcuts, API client
src/lib/avatar	clip format, sequence player, Mixamo rig mapping, idle and smoothing layer, non-manual markers, sentence planning,

Folder	Responsibility
	loader
src/components/avatar	the 3D viewer, lights, backdrop and the avatar model component
public	self-hosted MediaPipe files, avatar model and clips (the last two are git-ignored)

5.7 Storage

There is no database. Table 7 lists what is stored and where.

Table 7. Files used as storage.

Content	Location	In git
Trained weights, class list, threshold	backend/models	no
Sign clips (JSON)	frontend/public/animations	no (see Chapter 10)
Avatar model	frontend/public/avatar	no
Gloss dictionary, translation rules, sign registry	backend/config	yes
Processed INCLUDE features and manifests	data/processed	no
Recorded takes (landmarks only)	data/recorded	no
Evaluation results	docs/evaluation	yes

5.8 Application programming interface

The API is described in Appendix B. Every response carries an X-Process-Time-Ms header with the server time, requests with a body above 2 MB are refused with status 413, and the recorder endpoints answer 403 when recording is disabled by configuration.

6 Dataset

6.1 The INCLUDE dataset

INCLUDE is a word-level Indian Sign Language dataset published by Sridhar, Ganesan, Kumar and Khapra at ACM Multimedia 2020 [2]. The authors' lists name 4,292 videos in 263 classes grouped by category, with a predefined split into training, validation and test. The data is distributed as zip archives on Zenodo (record 4010759). This project downloaded the archives with a resumable streaming tool, processed each archive exactly once, and audited the result.

6.2 Processing and audit

Every video was converted into a sequence of 225 landmark values per frame (Section 6.4), taking every second frame. The audit (`backend/training/audit_include.py`, read-only on the data) checks the integrity of the result and its match to the authors' lists. Its overall result is PASS, with one documented exception.

Table 8. Audit of the processed INCLUDE data.

Check	Result
Videos listed by the authors, processed	4,292 listed, 4,284 processed (99.81 percent)
Manifest rows with a readable feature file	4,284 of 4,284
Missing, unreadable, empty arrays or wrong dimensions	0
NaN or infinite values	0
Duplicate manifest paths	0
Byte-identical feature arrays across files	0 groups
Videos in two splits, or in the wrong split	0
Classes with a sample	262 of 263 listed
Sequence length (frames)	minimum 17, median 31, 95th percentile 47, maximum 77
Frames without a left hand, without a right hand, without both	19.6, 29.6 and 11.9 percent
Samples per class	4 to 25, median 15

The eight videos that are not processed all belong to one class, "Second (Number)". The released archives contain no folder of that name (they hold only "Second"), and no file with those names exists in any archive, so this is a defect of the released lists and not of the processing. The consequence is that the class has no sample and the project uses 262 of the 263 listed classes. Face landmarks are not used.

6.3 Split

The authors' predefined split is kept exactly and is never re-split.

Table 9. Samples per split.

Split	Listed	Processed	Not processed
Training	3,127	3,121	6
Validation	348	347	1
Test	817	816	1

Sixteen classes have no validation sample because the authors' lists give them none, so macro metrics on validation average over 246 classes. No class lacks a test sample or a training sample (the smallest has 3 training samples). INCLUDE publishes no signer identifiers, so the split cannot be checked for signer overlap. **Every result in this report is on the authors' predefined split, signer independence not verified**, and absolute accuracies are probably optimistic for new signers.

6.4 Preprocessing and normalization

Landmarks come from MediaPipe Holistic (Python 0.10.14, model complexity 0) for the dataset, and from the WebAssembly build of the same family in the browser. Each frame contains 33 pose landmarks, 21 left-hand and 21 right-hand landmarks with coordinates x, y and z, which gives 75 x 3 = 225 values.

The normalization removes camera position and signer distance. With the left and right shoulder points s_L and s_R, the origin is their midpoint and the scale is the two-dimensional distance between them, so each landmark p becomes (p - origin) / scale. A missing hand is an all-zero block, and a frame without a usable pose is invalid and dropped. A short gap of at most three frames in a hand block is filled by linear interpolation when a hand is detected on both sides; longer gaps and gaps at the ends stay zero, so a hand that is absent is never invented. Finally the sequence is resampled linearly to 32 frames. The same functions serve training, the REST endpoints and the WebSocket, and the live path was checked to reproduce the offline predictions of 80 test samples exactly (within 1e-4).

Figure 5 shows one stored test sample drawn only from its features, to illustrate what the classifier sees. It contains no image of any person.

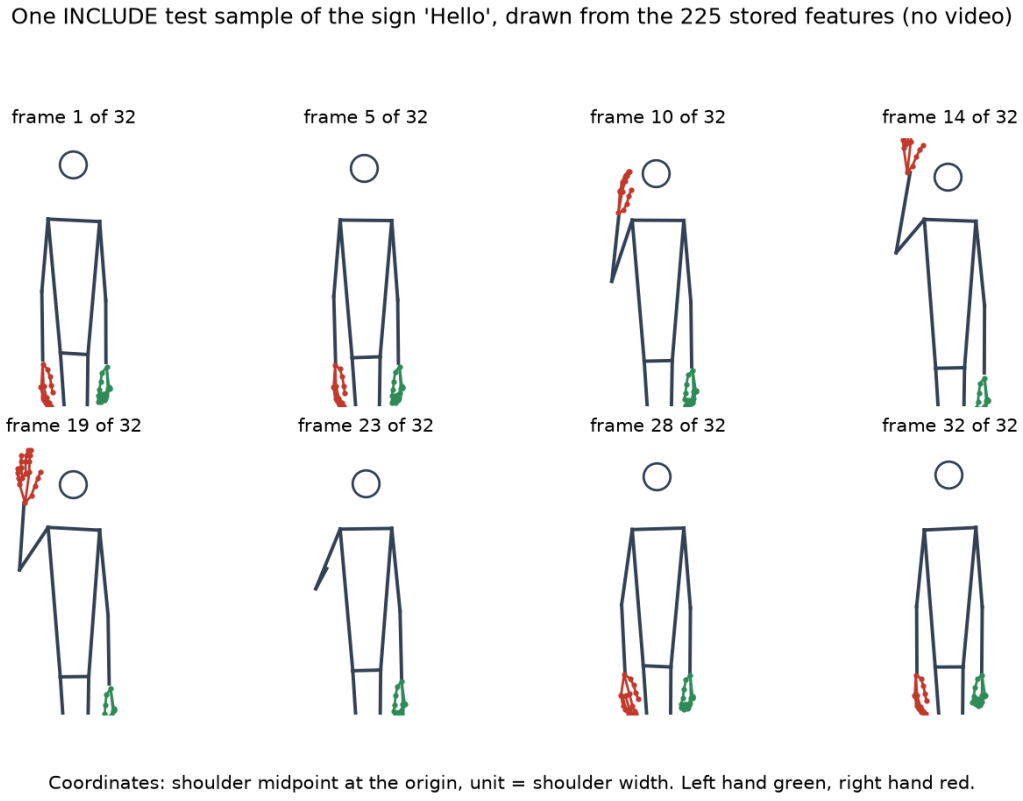


Figure 5. One INCLUDE test sample of the sign Hello drawn from its 225 stored features at eight of its 32 resampled frames. Only landmarks are drawn; the circle marks the nose position and is not a face.

Figure 6 summarizes the offline pipeline from the archives to the trained models.

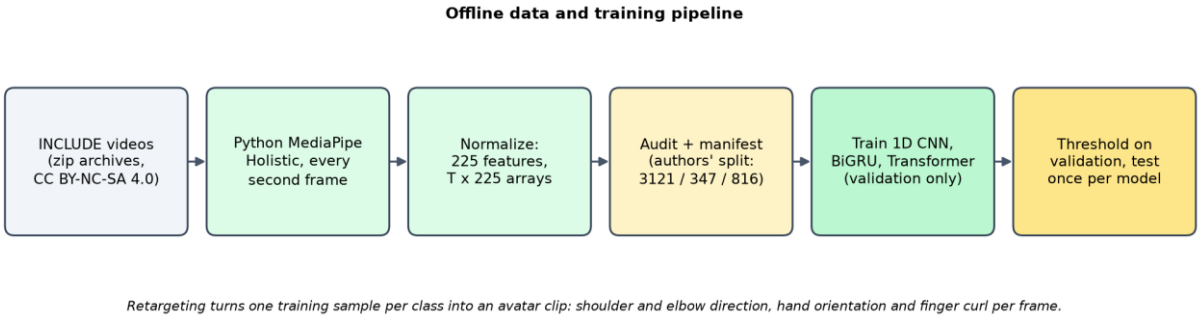


Figure 6. The offline data and training pipeline.

6.5 Licence

The dataset's own licence is stated inconsistently: CC BY 4.0 in the Zenodo record and CC BY-NC-SA 4.0 on a third-party page. The project follows the stricter reading. INCLUDE-derived material (features, statistics, models and clips) may be shared only non-commercially under CC BY-NC-SA 4.0 with attribution, and the data, the features, the trained weights and the clips are kept out of the public repository (`DATA_NOTICE.md`). The code is under the MIT licence, which does not cover INCLUDE or anything derived from it. Eleven early clips remain in the history of the git repository. The avatar model is a Mixamo asset whose terms have not been confirmed; it is not distributed.

6.6 What the dataset cannot show

The labels are the authors' class folders and no sign was checked by an ISL signer. The recordings come from one capture setup, so they do not show the variety of cameras, lighting and signers of real use. These are the reasons for the browser-path benchmark of Chapter 9 and for the recorder of Section 7.11.

7 Implementation

7.1 Recognition models

Three architectures are implemented behind one model builder and trained with identical settings.

Table 10. The three recognition models.

Model	Structure	Parameters
1D CNN	three Conv1d blocks (kernel sizes 5, 5 and 3, 128 channels, batch normalization, ReLU) over time, global mean and max pooling, dropout, linear head	343,558
BiGRU	two-layer bidirectional GRU (hidden size 128), mean over time, dropout, linear head	636,422
Transformer	linear projection to 128, learned positional embedding, two self-attention encoder layers (4 heads, feed-forward size 256), layer norm, mean pooling, linear head	360,710

The training settings are the same for all three: at most 60 epochs, patience 10 on the validation macro-F1, batch size 16, Adam with learning rate 0.001, hidden size 128, two layers (four heads for the Transformer), dropout 0.3, sequence length 32, hand-gap interpolation of three frames, seed 0. Every batch also receives Gaussian noise of standard deviation 0.01 on all 225 input values; the values are in shoulder widths, so this is about 0.0013 of the picture width at the median INCLUDE shoulder width of 0.133. Model selection and early stopping use the validation split only, and each model is evaluated on the test split once. The training pipeline refuses to train on a dataset that has not passed validation.

7.2 Training pipeline, augmentation and checkpoints

The training code is separate from the API; the API only loads the artifacts (weights, class list, threshold). Two additions were made for the deployment study.

- **Deployment augmentation** (`training/augment.py`): a vertical scale drawn from 0.85 to 1.20 (applied to 80 percent of samples and exact: the coordinates are re-normalized with the new shoulder distance, tested against re-normalizing scaled raw landmarks), landmark noise on x and y of detected landmarks with a per-sample standard deviation of up to 0.01 of the picture size, and hand dropout with a per-sample probability of up to 0.2 per frame and hand. Missing hands stay exactly zero.
- **Checkpoints and resume**: the training writes the last and the best state after every epoch (atomically) and can resume with the same settings; a run resumed after a stop ends with the same weights as an uninterrupted run (tested).

The resampling function was vectorized (about 30 times faster, identical values to 1e-6, tested), which made training with augmentation practical.

7.3 Live recognition and the threshold

The server loads the model once at start. A prediction below the threshold is returned as UNRECOGNIZED and leaves the sentence unchanged. The threshold is the lowest value of 0, 0.3, 0.5, 0.7, 0.8, 0.9 and 0.95 that gives at least 95 percent accuracy on the answered validation samples of the best model; for the 1D CNN this is 0.7 (validation coverage 0.8818, accuracy on answered

0.9608). It is stored with the model and sent to the client in the ready message together with its source. Through the live socket, 80 real test samples rebuilt into landmark frames gave exactly the offline model's labels and confidences.

7.4 Continuous signing

The segmenter (`app/recognition/segmenter.py`) works on the normalized feature rows. A frame is engaged when a wrist is above the rest line (2.0 shoulder widths below the shoulder midpoint; resting hands sit at about 2.8) and the wrists moved in the last 10 frames. A sign opens after 2 engaged frames and closes after 6 rest frames, where hands that are raised but still for 10 frames also count as rest. The segment keeps 5 frames before and 4 after, because the training samples begin and end with the hands down, and segments with fewer than 6 engaged frames are dropped as noise. Each segment is classified by the same model and threshold. The parameters were tuned on validation streams and then run once on test streams (Section 9.5). Signs signed back to back without a pause cannot be separated by this method.

7.5 Text to gloss

The rule layer (`backend/nlp`) uses spaCy 3.8.7 with the small English model. For each clause it outputs, in order: time signs, subject and other content in English order (a possessive uses the pronoun sign, so "my brother" gives I BROTHER), verbs after their objects and lemmatized (went becomes GO), NOT after the verbs, modals, and question words last. It drops and reports articles, the copula and auxiliaries, prepositions, the infinitive "to", "and", punctuation and intensifiers. A word that is not in the dictionary becomes a fingerspelling token such as FS:RAVI; no fingerspelling clips exist. Tags for negation, yes-no questions and WH questions drive a head movement of the avatar. The dictionary has 313 entries: the 263 INCLUDE class names and 50 grammar words that INCLUDE lacks. **These rules are simplified assumptions that no ISL signer or linguist has validated**; the tests document the rules and are not linguistic ground truth.

7.6 Gloss to English

The reverse direction is a heuristic: it reverses the order, inserts articles, the copula and auxiliaries, adds "to the" after motion verbs and place nouns, conjugates verbs from time signs (past after YESTERDAY or BEFORE, future after TOMORROW or NEXT, otherwise present), turns "pronoun NAME name" into "My name is X", and writes fingerspelled tokens as names. It has a small verb list, no plural handling and inserts "the" heuristically. Forty tested gloss sequences cover cases such as was, were, will be, on Monday, in the morning, possessives and greetings.

7.7 Retargeting landmarks to avatar clips

Clips are produced from INCLUDE landmark samples (`backend/retarget`). For each frame the converter computes the shoulder and elbow direction, the forearm direction, the hand orientation (wrist to middle knuckle and index to pinky knuckle) and one curl angle per finger. It then smooths the result, handles hand dropout (hold briefly, then relax to rest) and adds a 0.25 second ramp from and to the rest pose. Sixteen unit tests check the retargeter with forward kinematics under the same Euler convention as the avatar, including a left and right mirror test. Known limits are that the roll of the upper arm and forearm is not recovered, finger spread and thumb opposition are not modelled, and the depth coordinate is noisy. Each clip is registered with its provenance, and the registry cannot mark a

sign verified without a recorded human review. **All 262 clips are automatic retargets of one training sample each and none is reviewed by a signer.**

7.8 The avatar

The avatar is a Mixamo character loaded as a GLB file and driven by SignBridge clips. The rig mapping derives the arms-down pose from the loaded skeleton (not assumed), spreads the finger curl over the three joints of each finger, and rests bones without a clip track in that pose. Playback is by a sequence player: clips run back to back with 0.25 second eased transitions (smoothstep weights and quaternion slerp for body bones), speed control, pause, replay and restart. A presentation layer sits between the player and the skeleton. It adds a very small breathing, weight shift and head drift; relaxes an arm that hangs at rest (slight angle out, elbow bend, softly curled and fanned fingers) and fades that out completely as soon as the arm is lifted, so a sign is played as it was retargeted; and smooths the final pose over a time constant of 0.07 seconds so that nothing snaps at the start of a sentence, between clips or at the end. The scene has three-point lighting, a gradient backdrop and a camera framed on the upper body. Figure 7 shows the avatar across one sentence. If the GLB file is missing, a built-in procedural figure is used.



Figure 7. The avatar across the sentence "Hello. Thank you. Good morning. Doctor." at half speed, eight frames from the production build. The avatar is a mannequin; its face is not animated, and no clip is reviewed by a signer.

7.9 Frontend and user interface

The frontend is a Next.js 16 application with five tabs and a footer that carries the notices about research status, unreviewed clips, the INCLUDE attribution, the CC BY-NC-SA 4.0 non-commercial licence and the fact that video stays in the browser. The screenshots below were taken from the production build with a fake camera that plays an INCLUDE test video whose head is hidden by a mosaic, so that no face appears. The recognized sign shown in Figure 9 is the one the video shows.

- **Conversation** (Figure 8): one chat log for both directions. Recognized signs build a sentence that is sent as a message by a button, by a key, or after 3 seconds of rest. Typed messages are converted to gloss, signed by the avatar and kept in the chat with a line stating which words had no clip.
- **Sign to English** (Figure 9): the camera with the landmark overlay and the framing guide, the buffer counters, isolated recognition on request and a continuous mode.
- **English to Sign** (Figure 10): the gloss with the status of each clip, the avatar mid-sign, and playback controls. The supported vocabulary below the avatar lists which glosses have a clip.

- **Record sign** (Figure 11): a single-sign mode and a guided session (Section 7.11).
- **Landmark check** (Figure 12): records 4 seconds of video with the browser landmarks so that Python MediaPipe can be compared offline.

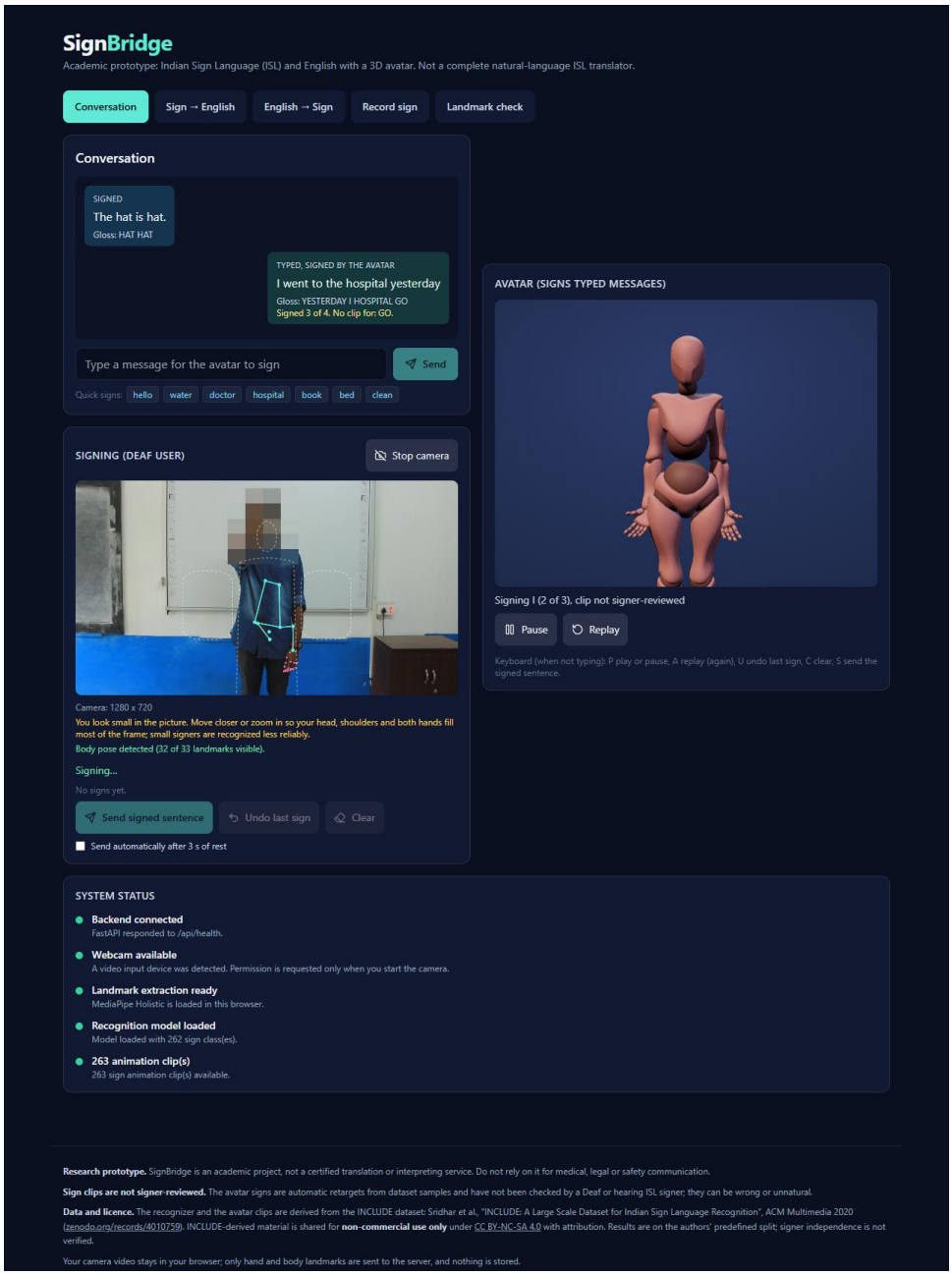


Figure 8. Conversation tab: a signed message, a typed message that the avatar is signing, the camera with its overlay and the system status. The camera feed is an INCLUDE test video with the head masked.

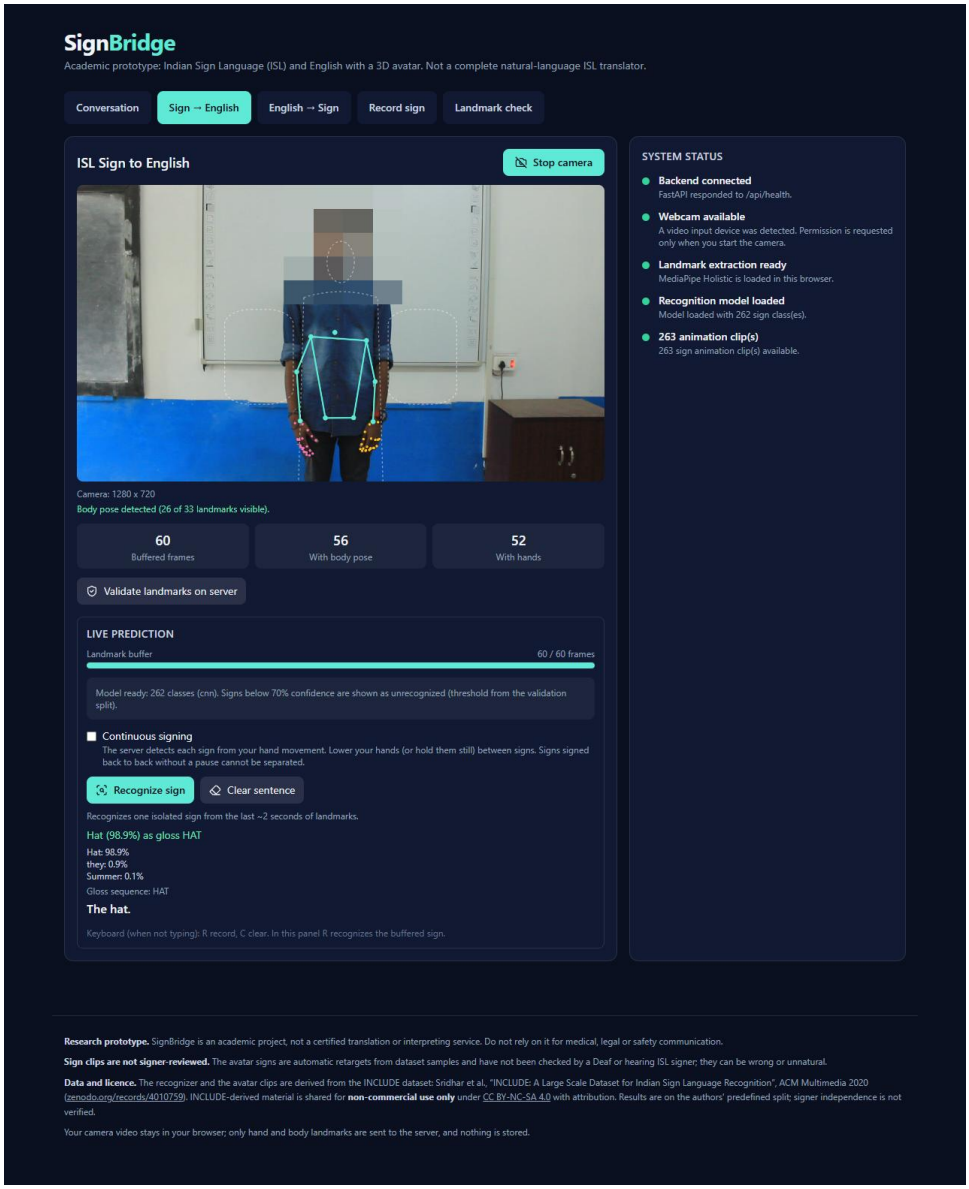


Figure 9**.** Sign to English tab with the fake camera: landmark overlay, the dashed framing guide, the buffer counters and an isolated recognition result.

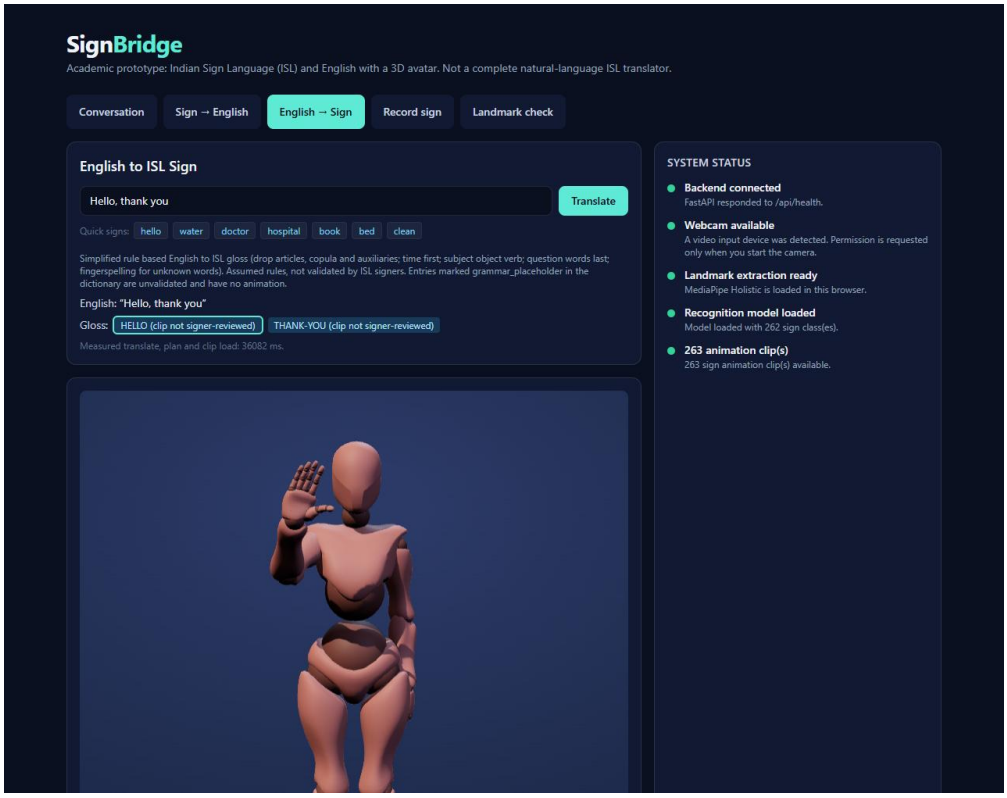


Figure 10**.** English to Sign tab: the gloss of "Hello, thank you", the avatar in the middle of the sign HELLO and the playback controls. The page continues with the vocabulary list.

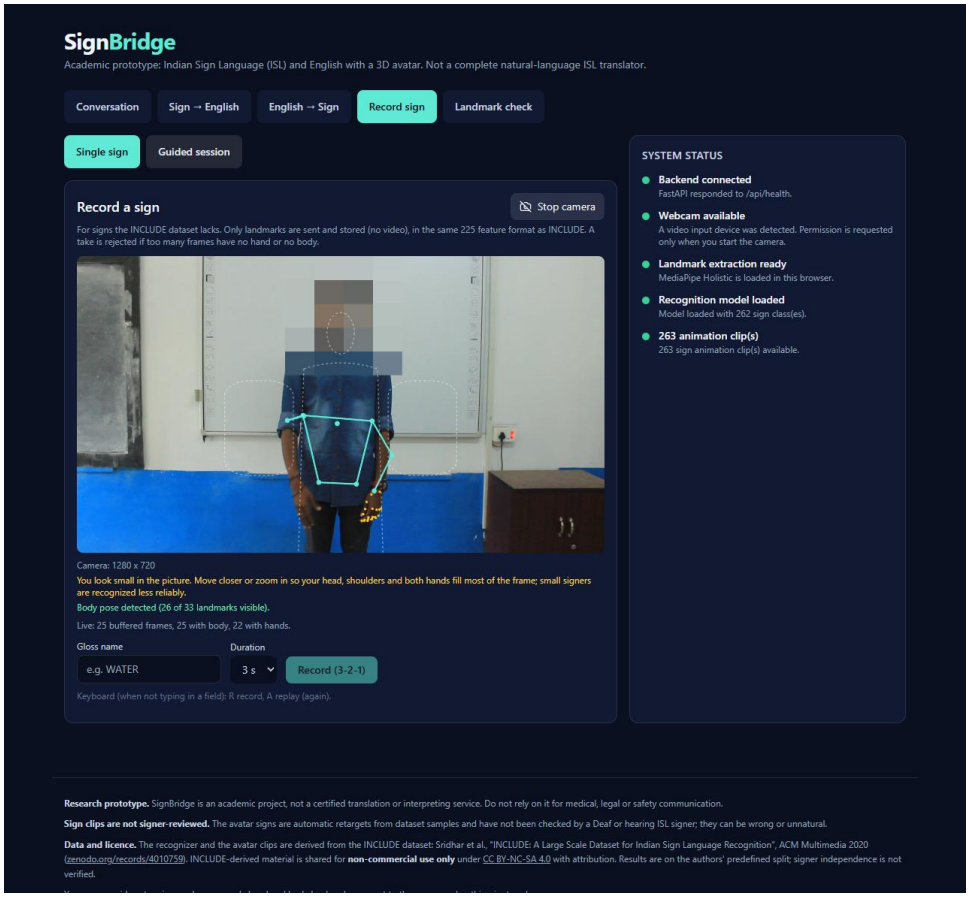


Figure 11**.** Record sign tab (single sign mode) with the camera running. Nothing is recorded until the user presses the record button.

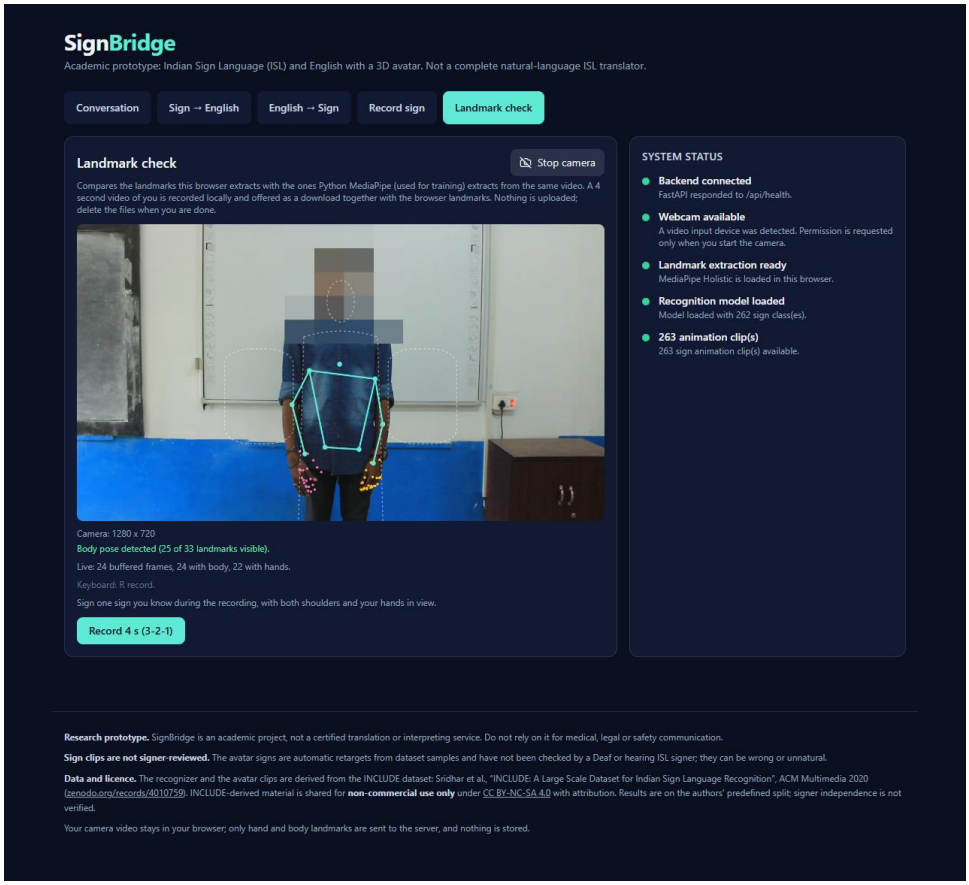


Figure 12. Landmark check tab with the camera running. It records a short video and the browser landmarks locally for the offline comparison.

The camera is requested at 1280 x 720 (an ideal constraint, so the nearest size is used) with a fall back to whatever the camera gives, and the real resolution is shown. MediaPipe returns x as a fraction of the picture width and y as a fraction of its height, and the dataset videos are 16:9, so a 4:3 picture gives vertical distances that are smaller by 0.75 after normalization. The browser therefore multiplies every vertical coordinate by (16/9) divided by the picture aspect (4/3 for a 4:3 camera) before it is sent. The framing guide draws the head, shoulders and hands at a shoulder width of about 14 percent of the picture width, inside the tested range, and the app warns when the smoothed shoulder width falls below 0.10 (Section 9.3).

7.10 Backend

The backend is a FastAPI application. Settings are read from SIGNBRIDGE_ environment variables: the allowed web origins (and a pattern for preview deployments), a switch that makes the WebSocket check the origin, a switch that turns the recorder off, and the model and clip folders. The body size is limited to 2 MB. The WebSocket handler keeps one session per connection with the sentence and the segmenter state. Startup loads the model and preloads spaCy, and the model status endpoint reports not trained, loaded or error without ever substituting a prediction.

7.11 Recorder

The recorder collects landmarks only, never video. In single mode the user records a sign that INCLUDE lacks after a 3-2-1 countdown. In guided mode the user enters a signer identifier, the number of takes per sign and a list of signs (the 50 grammar words are preloaded, with WHERE, GO, LIKE, NOT, HELP, WHAT, YES and NO first); the session shows one sign at a time, counts only

accepted takes, explains rejected ones and resumes for the same identifier. A take is accepted when it has at least 20 frames, a frame rate of 5 to 60 per second, a body in at least 90 percent of the frames, a hand in at least 70 percent, and the segmenter finds a sign in it. Takes are stored as landmark arrays under the signer and the sign, and a promotion tool turns a good single take into a clip. The signer roles for the self-recorded data are fixed in advance (training S01 to S03, validation S04, test S05), and a builder produces a combined manifest with a signer-level split.

7.12 Deployment

A production build is served locally with one script. Two deployments are prepared and tested locally but were not performed: the web application on Vercel and the API in a Docker container for Hugging Face Spaces. The container uses CPU PyTorch without MediaPipe or OpenCV, runs as an unprivileged user on port 7860 and downloads the weights and clips at start from a private repository, so that they are never in the image or in git. It was built and run locally (about 1.6 GB on disk), and health, the CORS header for an allowed origin, a correct prediction through the WebSocket, a refused foreign origin and a disabled recorder were checked. In the public configuration the recorder tab is hidden and the API refuses recording. The step-by-step instructions are in `docs/DEPLOYMENT.md`. Section 10.2 lists the problems that were found in later commits connected with the Vercel deployment, one of which has been fixed.

8 Testing

8.1 Test strategy

Testing has four layers: automated unit and integration tests, a benchmark of the complete browser path, end-to-end runs in a real browser with a fake camera, and manual inspection. The automated tests use tiny synthetic models and synthetic landmarks, so their scores are never reported as results. They check the mechanics of the code. The measured results of the project come from the evaluation scripts (Chapter 9).

8.2 Automated tests

The test script runs the backend tests with pytest, the TypeScript compiler, ESLint, the frontend tests with Vitest and the production build. On the current repository head the backend suite has 367 tests and the frontend suite has 137 tests, and all of them pass; the TypeScript compiler, ESLint and the production build are clean.

Table 11. Automated test suites.

Suite	Tests	What it covers
Backend (pytest)	367 in 31 files	landmark normalization, preprocessing, models, training pipeline, inference, API, WebSocket, recorder, retargeting, text to gloss, gloss to English, registry, data tools, deployment switches, augmentation, checkpoint and resume
Frontend (Vitest)	137 in 23 files	landmark conversion and buffers, camera request and fallback, overlay and framing warning, WebSocket client, recorder, shortcuts, avatar player, rig mapping, idle and smoothing layer, non-manual markers, sentence planning, status logic, deployment switches, the HTTP fallback route (a failed upstream call never returns a label), honest status routes, registry and plan rules
Type check, lint, production build	pass	the whole frontend

Table 12. The largest backend test files.

File	Tests	Subject
test_nlp.py	80	text to gloss rules: 36 English to gloss pairs, 20 gloss to English pairs, 8 round trips and edge cases
test_gloss_to_text_quality.py	43	40 gloss sequences for the English generator and its quality rules
test_slr_pipeline.py	24	recognition pipeline, preprocessing and models on synthetic data
test_retarget.py	16	landmarks to bones with forward kinematics, mirror test, clip validity
test_translation.py	15	vocabulary and translation endpoints
test_training_pipeline.py	15	training, resume, checkpoints, splitting, metrics
test_record_session.py	14	guided sessions, plan, resume counts, quality check
test_clips.py	12	clip format validation and the avatar plan

Specific properties that are tested include: the live and offline preprocessing agree to 1e-6; the vectorized resampling equals the per-column interpolation to 1e-6; the vertical-scale augmentation equals re-normalizing scaled raw landmarks; a training run resumed from a checkpoint ends with the

same weights as an uninterrupted run; the recorder endpoints answer 403 when disabled and write nothing; a WebSocket from a foreign origin is refused when the origin check is on; and an unsupported or missing model never produces a prediction.

8.3 Integration tests

Integration tests drive the real API with the test client: the REST endpoints end to end, the WebSocket with isolated and continuous messages, and the recorder with synthetic takes. In addition, 80 real INCLUDE test samples were rebuilt into landmark frames and sent through the live socket; all 80 gave exactly the offline model's labels and confidences (identical within 1e-4).

8.4 Browser-path benchmark

The benchmark feeds INCLUDE videos frame by frame through the real in-browser MediaPipe Holistic of the prototype and classifies the result with the live code path. It is described in Section 9.3 and its limits are stated there. It uses frame stepping instead of a looping fake camera because headless Chrome extracts only about 6 frames per second and a live fake camera would drop frames and mix capture starvation into the error rate. The extracted landmark files of the larger corpora (211 validation and 312 test videos, two framings) were validated for readability, shape, finite values, duplicates and completeness before use.

8.5 End-to-end tests with a fake camera

Real Chrome was driven by Playwright with a fake camera that plays a converted INCLUDE test video. The first run used letterboxed 4:3 pictures of three test videos (Hat, Doctor, Yesterday), before the picture aspect rescale existed.

Table 13. End-to-end test with a fake camera before and after the aspect rescale (isolated and continuous modes, one run each).

Video	Before the rescale	After the rescale
Hat	recognized correctly	recognized correctly (99.6 percent isolated; HAT in continuous mode)
Doctor	a skirt sign	DOCTOR (95 percent) in continuous mode, but unrecognized when isolated and two SLOW detections in continuous mode
Yesterday	THEY	unrecognized (best guess Cell phone, 56 percent)

These three runs are a smoke test, not a measurement. They exposed the picture aspect problem and led to the benchmark of 100 videos. The guided session also rejected a take with only 18 frames with a clear reason, as designed. The same fake camera, with the head hidden, produced the screenshots of this report.

8.6 Manual verification

The production build was inspected in a browser: all five tabs, the landmark overlay, the camera resolution line, the framing guide, the avatar at rest and mid-sign, the chat in both directions and the footer notices. The layout was checked down to a 375 pixel wide screen.

8.7 What testing does not show

No test used a real webcam with a real signer. The automated tests use synthetic models and landmarks, the benchmark uses dataset videos re-encoded as pictures, and the fake-camera runs use three videos. No clip and no text rule has been checked by an ISL signer, so the tests prove consistency, not linguistic correctness.

9 Results

All numbers in this chapter come from `docs/evaluation` and are on the authors' predefined split of INCLUDE, signer independence not verified. Numbers that need the three-seed runs are marked **PENDING** in red; they are filled automatically when the file `docs/evaluation/three_seed_summary.json` exists and the report is rebuilt.

9.1 Model comparison

The three models were trained with identical settings and each was evaluated once on the test split (816 samples, 262 classes). The 1D CNN is clearly the best and is the model used live.

Table 14. Test results of the baseline models (seed 0).

Model	Epochs run	Validation macro-F1	Test top-1	Test top-5	Test macro-F1	Macro precision	Macro recall
1D CNN	46	0.9039	0.9130	0.9902	0.9107	0.9307	0.9176
BiGRU	38	0.7832	0.7929	0.9730	0.7869	0.8205	0.8072
Transformer	43	0.7128	0.7328	0.9596	0.7100	0.7512	0.7364

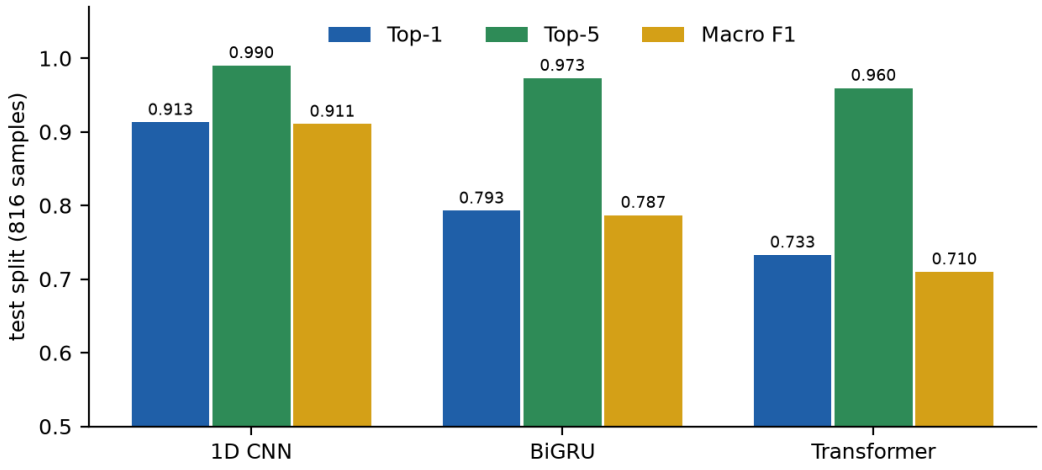


Figure 13. Test results of the three baseline models.

For the 1D CNN, 212 of the 262 classes are classified perfectly on the test split, but many classes have only one to four test samples, so per-class accuracies are very coarse: the lowest are Afternoon, Bill and Sunday (0 of 1, 0 of 3 and 0 of 1 samples) and Blue, Brother and Husband (one of four each). Figure 14 and Figure 15 show the most frequent confusions. They are pairs that are close in meaning or in form, such as Brother and Sister, Husband and Wife, Boat and Fish, and I and you. These counts are small (two or three samples each), so they illustrate the kind of error and should not be ranked.

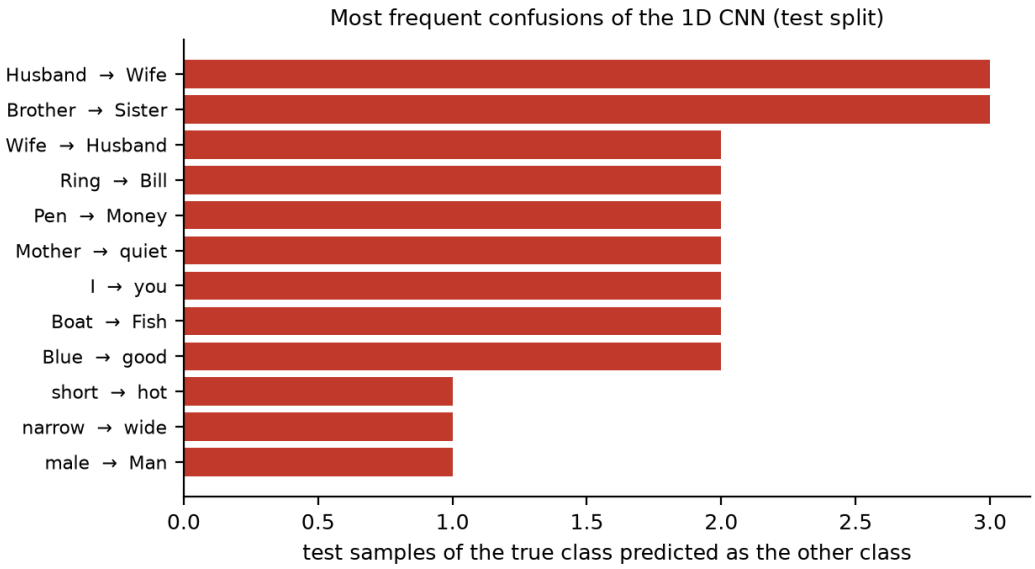


Figure 14. The most frequent confusions of the 1D CNN on the test split.

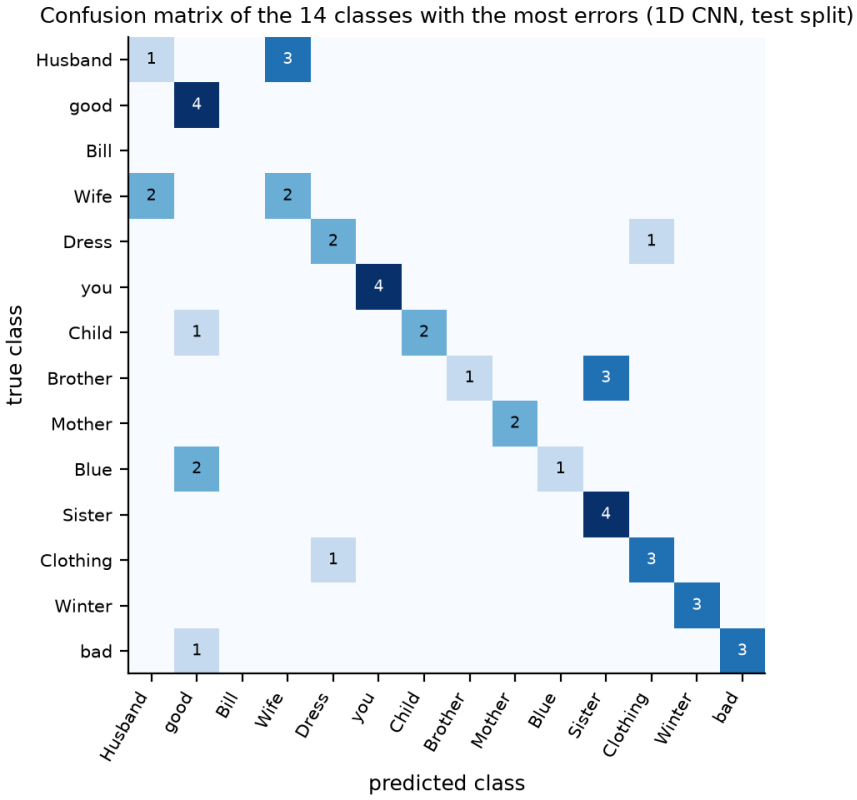


Figure 15. Confusion matrix restricted to the 14 classes with the most errors (1D CNN, test split).

9.2 Threshold trade-off

A prediction whose top confidence is below the threshold is reported as UNRECOGNIZED, which trades coverage for accuracy. Table 15 gives the test-split trade-off at three thresholds and Figure 16 shows the curves for all of them. The live threshold of 0.7 was chosen on the validation split of the CNN (at least 95 percent accuracy on answered samples). On the test split it answers 0.8676 of the samples with an accuracy of 0.9548 on the answered ones.

Table 15. Coverage and accuracy on answered samples against the confidence threshold (test split, baseline models, seed 0).

Model	Threshold 0.5 (coverage, accuracy)	Threshold 0.7 (coverage, accuracy)	Threshold 0.9 (coverage, accuracy)
1D CNN	0.9559, 0.9346	0.8676, 0.9548	0.7451, 0.9770
BiGRU	0.8885, 0.8455	0.7279, 0.9074	0.5159, 0.9667
Transformer	0.9289, 0.7665	0.7549, 0.8279	0.5404, 0.9161

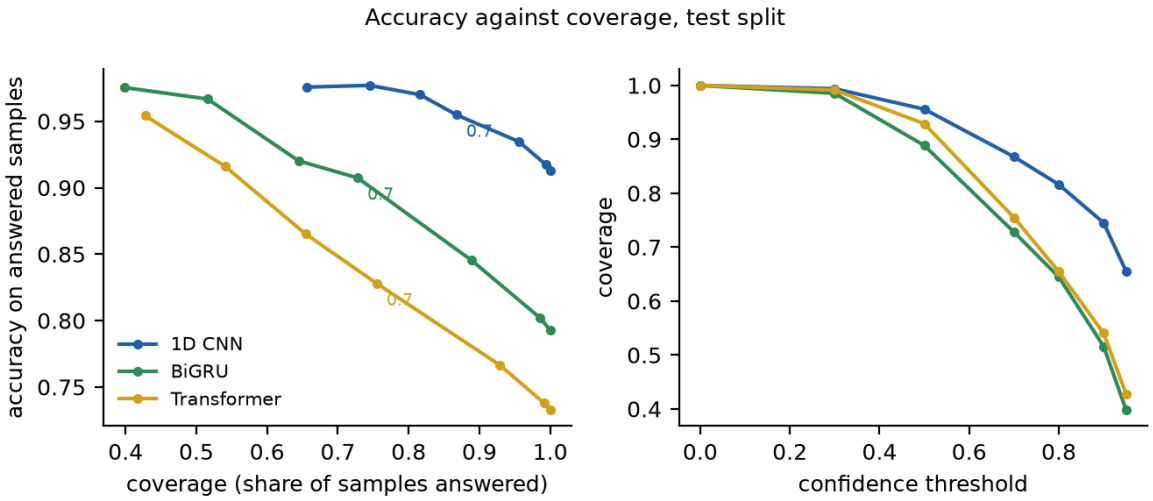


Figure 16. Accuracy against coverage (left) and coverage against the threshold (right) for the three baseline models on the test split. The mark 0.7 is the live threshold.

9.3 Deployment-gap results

9.3.1 Browser path against the offline path

To measure what a deployment changes, 100 test videos (one per class, drawn with a fixed seed from the test samples whose video is available) were passed through the real in-browser MediaPipe frame by frame, in 16:9 (1280 x 720) and in 4:3 (letterboxed into 640 x 480), and classified with the live code path by the 1D CNN. Accuracy is the share of videos answered above the threshold and correctly; wrong accepted is answered above the threshold and wrongly; the rest is UNRECOGNIZED. Intervals are 95 percent Wilson intervals; paired comparisons use the exact McNemar test.

Table 16. The 1D CNN on 100 test videos at threshold 0.7.

Source	Accuracy	95 percent interval	Wrong accepted	Unrecognized	Top-1
Offline features	0.82	0.73 to 0.88	0.03	0.15	0.92
Python MediaPipe, 16:9	0.81	0.72 to 0.87	0.03	0.16	0.90
Browser, 16:9	0.78	0.69 to 0.85	0.02	0.20	0.87
Python MediaPipe, 4:3	0.70	0.60 to 0.78	0.05	0.25	0.81
Browser, 4:3 with the vertical rescale	0.65	0.55 to 0.74	0.07	0.28	0.78
Python MediaPipe, 4:3 without the rescale	0.08	not computed	0.29	0.63	0.12
Browser, 4:3 without the rescale	0.07	not computed	0.29	0.64	0.10

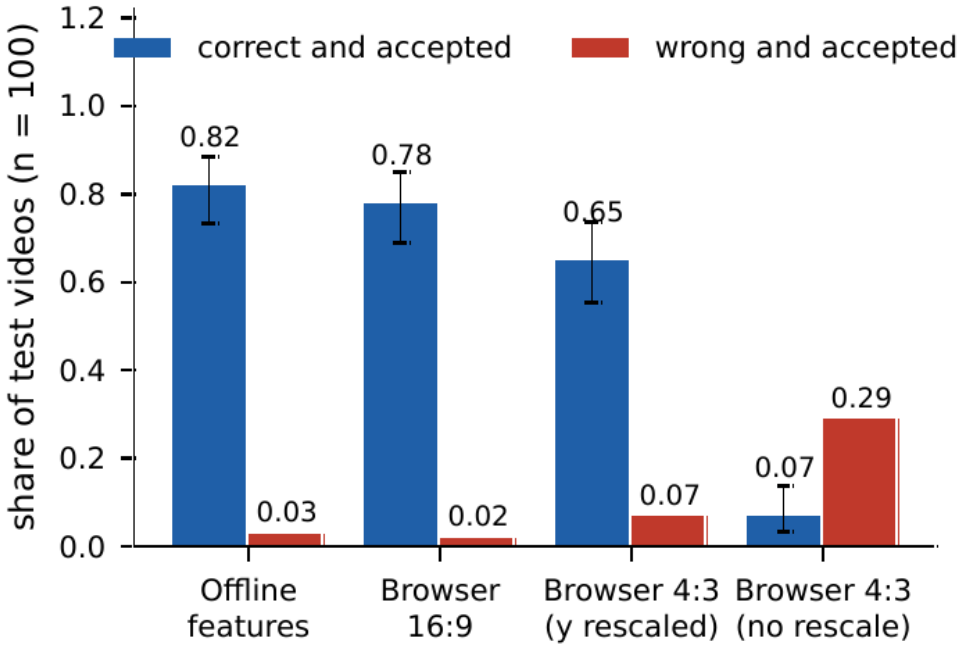


Figure 17. The 1D CNN on 100 test videos at threshold 0.7: correct-and-accepted share with 95 percent Wilson intervals and wrong-and-accepted share, offline and in the browser path.

The results lead to four findings. In native 16:9 the browser path is within noise of the offline path (0.78 against 0.82; 6 videos lost and 2 gained, exact McNemar p = 0.29). In 4:3 the loss is large (0.65; 22 videos lost and 5 gained, p = 0.002) and wrong answers increase from 0.03 to 0.07. On identical frames the browser is 3 to 5 points below Python MediaPipe, which is not significant at this sample size, so the extractor is a minor contributor and the picture is the major one. Without the vertical rescale, 4:3 input gives 0.07 accuracy with 29 percent confident wrong answers: the model fails by answering wrongly, not by hesitating. A 16:9 picture resized to 640 x 360 gives 0.80 and a centre crop of the 16:9 frame to 4:3 (a webcam-like fill) with the rescale gives 0.76.

The loss that remains in 4:3 after the exact rescale (0.65 against 0.78 in 16:9 on the test videos, 0.58 against 0.75 on the validation videos) is **not explained** by anything that was measured. It is not a vertical scale error, because the rescale is exact. It is not signer size, because letterboxing keeps the picture width. It is not resolution, because the smaller 16:9 picture scores 0.80. A possible cause is that the black padding of the letterbox disturbs MediaPipe; this is an **untested hypothesis**, and a padded 16:9 picture and an unpadded 4:3 crop of identical content would test it.

9.3.2 Vertical scale, noise and hand dropout

Controlled perturbations were applied to the browser landmarks of 100 validation videos (Figure 18 and Table 17). Without a perturbation the CNN scores 0.75 in 16:9 and 0.58 in 4:3. A vertical scale error of 10 percent lowers the 16:9 accuracy to 0.43 (factor 0.9) or 0.65 (factor 1.1); an error of 25 percent leaves 0.26 with 36 percent wrong answers, and a factor of 0.75, which is exactly what an unrescaled 4:3 camera produces, leaves 0.05. The model is therefore very sensitive to a cue that normalization by shoulder width cannot remove. Noise and hand dropout cost accuracy gradually, whereas vertical scale is a cliff.

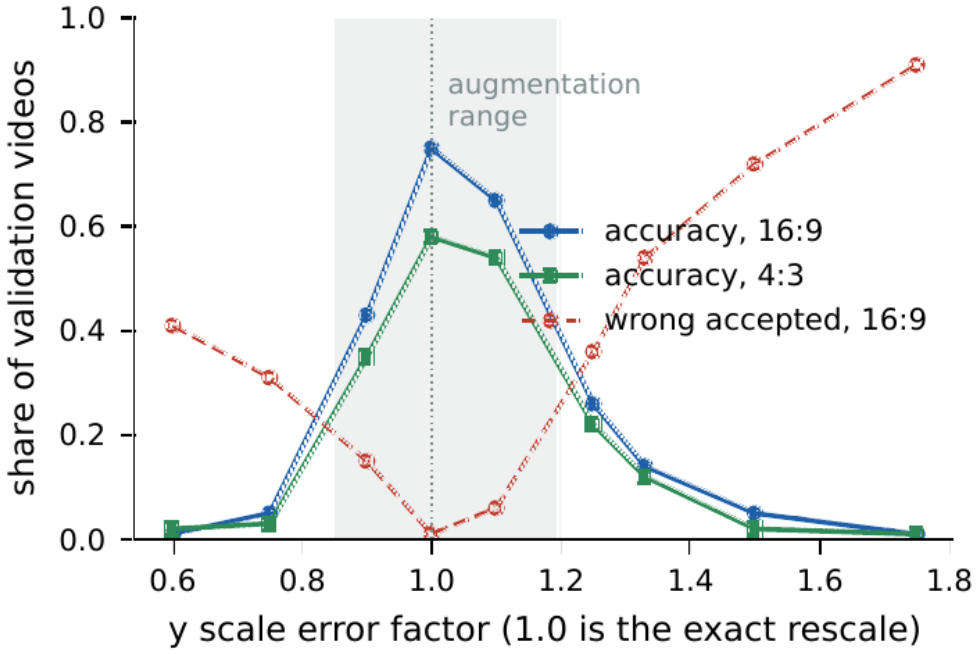


Figure 18. The 1D CNN against a vertical scale error applied to browser landmarks of 100 validation videos. The shaded band is the range of the augmentation.

Table 17. The 1D CNN on browser 16:9 landmarks of 100 validation videos with added noise or removed hands, threshold 0.7.

Noise (picture fraction)	Accuracy	Wrong accepted	Hand dropout (share of frames)	Accuracy	Wrong accepted
0.002	0.74	0.02	0.1	0.77	0.02
0.005	0.72	0.03	0.2	0.69	0.04
0.01	0.70	0.06	0.4	0.55	0.09
0.02	0.44	0.11	0.6	0.30	0.16

9.3.3 Signer size in the picture

Thirty validation videos were shrunk toward the centre of the picture and classified with Python MediaPipe, binned by the median shoulder width as a fraction of the picture width (Table 18 and Figure 19). INCLUDE signers measure 0.116 to 0.149 (median 0.133). Accuracy is 0.89 for widths of 0.13 to 0.17 and 0.83 for 0.10 to 0.13, falls to 0.71 for 0.08 to 0.10 and to 0.59 and 0.50 below 0.08, where 12 to 14 percent of the answers are confidently wrong. At 30 percent of the size no body pose was found. The application therefore warns when the smoothed shoulder width falls below 0.10.

Table 18. The 1D CNN by shoulder width, validation videos shrunk toward the centre (Python MediaPipe, threshold 0.7).

Shoulder width (fraction of picture width)	Accuracy	Wrong accepted	Video and size pairs
0.04 to 0.06	0.50	0.14	14
0.06 to 0.08	0.59	0.12	32
0.08 to 0.10	0.71	0.00	14
0.10 to 0.13	0.83	0.00	12
0.13 to 0.17	0.89	0.00	18

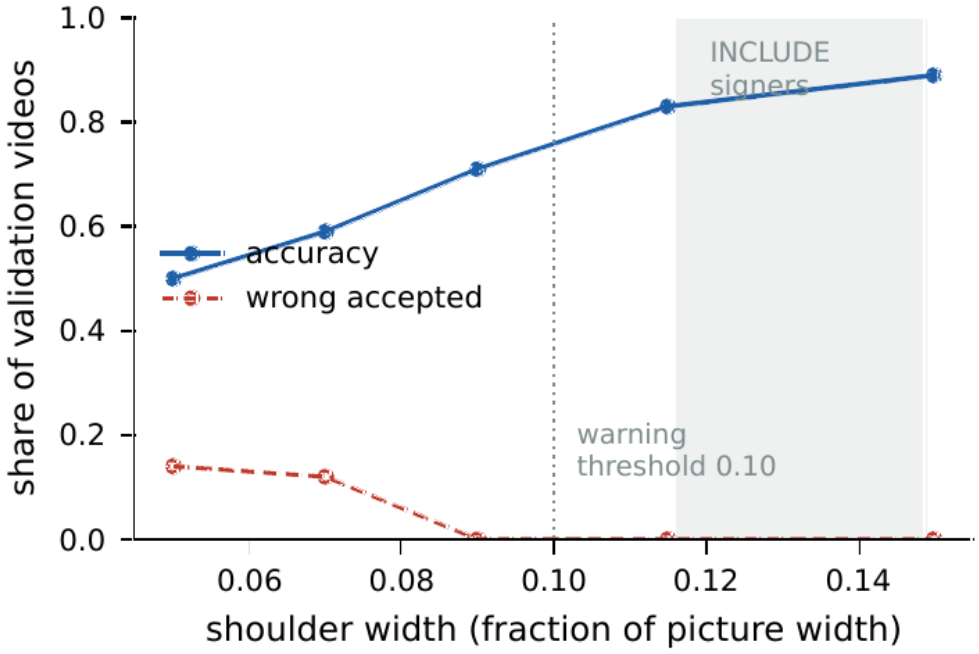


Figure 19. The 1D CNN against the median shoulder width, validation videos shrunk toward the centre. The dotted line is the warning threshold of the application.

Larger signers could not be tested by cropping. The same 30 videos were cropped toward the signer to 75, 60 and 50 percent of the picture (median shoulder widths 0.175, 0.216 and 0.259). Accuracy fell to 0.30, 0.37 and 0.43, but a hand was found in only 63, 36 and 33 percent of the frames against about 87 percent uncropped, because the crop cuts the signing space off. Size and lost hands cannot be separated this way, so sizes above 0.15 remain untested, and the framing guide of the application was therefore placed inside the tested range, with the shoulders at about 14 percent of the picture width.

9.4 Augmentation

The measurements above fixed the augmentation of Section 7.2: a vertical scale from 0.85 to 1.20, landmark noise up to 0.01 of the picture size and hand dropout up to 0.2. The three architectures were retrained with identical settings and the augmentation added, for one seed (seed 0), and each was evaluated once on the test split.

Table 19. Test results of the baseline and augmented models, single seed (seed 0). Differences between single runs are not separated from training variance.

Model	Top-1 baseline	Top-1 augmented	Macro-F1 baseline	Macro-F1 augmented
1D CNN	0.9130	0.9056	0.9107	0.9103
BiGRU	0.7929	0.8787	0.7869	0.8790
Transformer	0.7328	0.7806	0.7100	0.7731

The robustness of the augmented models to the deployment conditions is measured by the same browser-path benchmark and perturbation sweeps, on larger corpora (211 validation and 312 test videos in two framings) and with two more seeds per model. The report presents mean and spread over three seeds in Table 20. The raw per-video results of the first seed are in `docs/evaluation/eval_val.json` and `eval_test.json`.

Table 20. Three-seed summary, mean and spread over seeds 0, 1 and 2, accuracy at the validation-chosen threshold unless stated (authors' predefined split, signer independence not verified). Test videos are evaluated once per trained model.

Condition	CNN baseline	CNN augmented	BiGRU baseline	BiGRU augmented	Transformer baseline	Transformer augmented
Offline features	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
Browser, 16:9	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
Browser, 4:3 with rescale	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
Browser, 4:3 raw	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
Wrong-accepted rate, browser 16:9	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
Validation-chosen threshold	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING

The interpretation of the three-seed numbers is pending. Two points can already be stated. The augmentation cannot be expected to close the loss that remains in 4:3 after the exact rescale, because that loss is explained neither by vertical scale nor by signer size nor by resolution. And augmentation changes how confident a model is, so the validation-chosen threshold can differ between baseline and augmented models; the threshold column of the table reports it per model.

9.5 Segmentation

The continuous mode was evaluated on streams that join 3 to 5 real INCLUDE samples of different classes (80 streams per row, fixed seed). Between two samples a number of rest frames is inserted (a copy of the last frame of the earlier sample, still, hands down, with small noise); the samples already carry a short rest margin. The rest frames are synthetic and real people may pause differently. Ground truth is the span of each sample. The parameters were tuned on the validation streams (the rest line and the padding mattered) and the test streams were run once with the final settings.

Table 21. Segmentation results on streams of real INCLUDE samples (real signs split correctly, recognized correctly, accepted but wrong, UNRECOGNIZED).

Split	Rest frames between signs	Real signs	Split correctly	Recognized correctly	Accepted but wrong	UNRECOGNIZED	Whole sentence right
Validation	0 (natural margins only)	313	203 (64.9 percent)	161 (51.4 percent)	10 (3.2 percent)	32 (10.2 percent)	34 of 80
Validation	10	313	308 (98.4 percent)	250 (79.9 percent)	13 (4.2 percent)	45 (14.4 percent)	36 of 80
Validation	20	313	310 (99.0 percent)	252 (80.5 percent)	13 (4.2 percent)	45 (14.4 percent)	36 of 80
Test	0 (natural margins only)	322	242 (75.2 percent)	188 (58.4 percent)	20 (6.2 percent)	34 (10.6 percent)	28 of 80
Test	10	322	316 (98.1 percent)	247 (76.7 percent)	22 (6.8 percent)	47 (14.6 percent)	28 of 80
Test	20	322	320 (99.4 percent)	248 (77.0 percent)	23 (7.1 percent)	49 (15.2 percent)	28 of 80

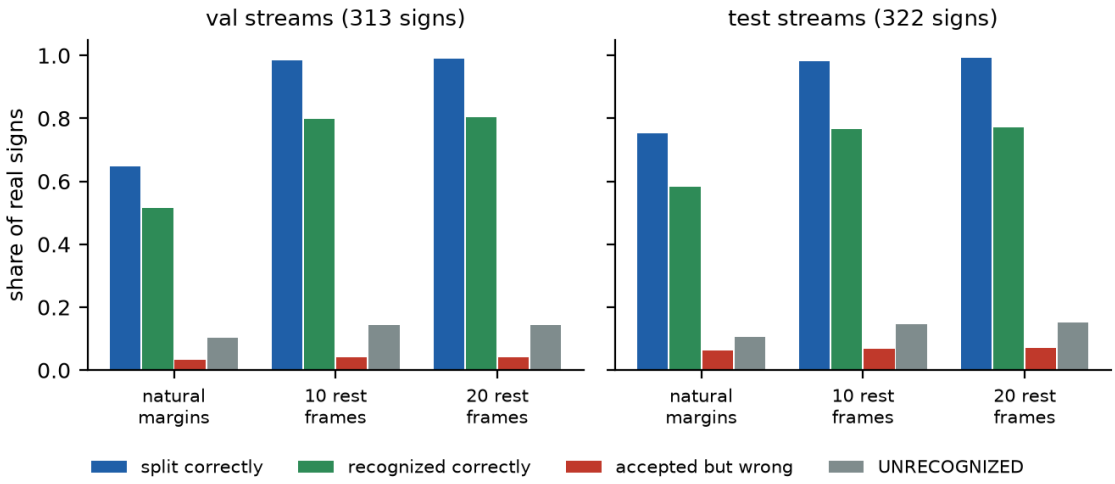


Figure 20**.** Segmentation results on validation and test streams for three amounts of rest between signs.

With 10 or more rest frames the segmenter finds nearly every sign (98 to 99 percent split correctly). With only the natural margins, 24 to 33 percent of the signs are merged with a neighbour or fragmented: signs signed back to back cannot be separated by this method. For comparison, the isolated mode on whole test samples accepts 86.8 percent at the live threshold with 95.5 percent of those correct, which is about 82.8 percent right, 3.9 percent wrong and 13.2 percent unrecognized. Segmentation costs about 6 points of correct recognition (77.0 against 82.8 percent) and nearly doubles the wrong accepted signs (7.1 against 3.9 percent), because the cut segment is not exactly the span that the model was trained on.

9.6 Comparison with published INCLUDE results

The accuracies of Table 14 cannot be compared directly with published work. Pose-based models on INCLUDE have been reported in the OpenHands study [5] and the INCLUDE paper itself [2], but those numbers have not been checked against the published versions for this report and are not repeated. The comparison would also be loose: this project uses 262 of the 263 classes, MediaPipe landmarks, its own training settings and one seed in the baseline, and the split is the authors' in every case, so none of the works can claim signer independence. The result is reported here as it is.

10 Limitations and Future Work

10.1 Limitations

This section states the limits of the work with the same honesty as the accompanying paper.

- **Authors' predefined split, signer independence not verified.** INCLUDE has no signer identifiers, so the same person may appear in training and test. Every accuracy in this report is therefore probably optimistic for a new signer, and the deployment gap measured here is a lower bound on the gap for people unseen in training.
- **No face features.** Only pose and hands are tracked. Facial expression carries grammatical information in Indian Sign Language, for example for questions and negation. The recognizer cannot use it and the avatar does not animate a face. The head movements for negation and questions are rule-based approximations that are not taken from signer-validated data.
- **Clips are not signer-reviewed.** All 262 clips are automatic retargets of one training sample each, with an approximate bone mapping and no recovered forearm roll or finger spread. No clip has been reviewed by a signer, the backend registry marks none as verified (an unsupported review claim for the WATER clip was found in a later commit and removed, Section 10.2), and some will look wrong or unnatural. The system is not an interpreting service.
- **Rules are assumptions.** The text to gloss rules and the gloss to English heuristics are simplified assumptions for single-clause sentences that no ISL signer or linguist has validated.
- **Vocabulary.** The recognizer knows 262 INCLUDE classes. Fifty grammar words (NOT, WHAT, WHERE, GO, HELP, YES, NO and others) can be typed and signed only if a clip exists; they have no recognition data and no clip, and fingerspelling clips do not exist. In total 262 of 313 dictionary glosses have a clip.
- **Real webcams are unmeasured.** The browser results use dataset videos re-encoded as pictures. No real webcam, real lighting, real user or left-handed signer is included, and the framing study covers only sizes up to the dataset range.
- **Residual 4:3 loss unexplained.** The loss that remains in 4:3 pictures after the exact rescale is not explained by vertical scale, signer size or resolution. Black padding affecting MediaPipe is an untested hypothesis.
- **Small samples.** One sample per class gives wide intervals (about plus or minus 9 points for 100 videos), many classes have only a few test samples, and the single-seed CNN study has no estimate of training variance.
- **Continuous signing.** Signs signed back to back without a pause cannot be separated, the pauses in the tests are synthetic, and the segmented accuracy is a few points below the isolated accuracy.
- **Licence.** The INCLUDE licence is stated inconsistently (CC BY 4.0 in the Zenodo record, CC BY-NC-SA 4.0 on a third-party page). The project follows the stricter reading: INCLUDE-derived material is non-commercial only, with attribution. Eleven early clips remain in the git history. The terms of the Mixamo avatar are unconfirmed, so the avatar file is not distributed and a public deployment should not include it until they are confirmed.
- **Not deployed.** The Vercel and Hugging Face deployments were prepared and the container was tested locally, but nothing was deployed, and the download of assets from a private repository was not tested against a real repository.

10.2 Problems found in commits made outside the project sessions

Commits made after the measurements of this report (by the repository owner's git identity, outside the sessions that wrote the rest of the history) added Next.js route handlers under `frontend/src/app/api` as a fallback for a Vercel deployment. They were reviewed on 3 November 2026 (`docs/COMMIT_REVIEW_2026-11-03.md`).

- **A fake fallback was found and removed.** The route `/api/signs/recognize` did not run the recognition model. When its call to an external Hugging Face Space failed (and that call sent a label, not the landmarks), it returned the label of the request or a fixed "Hello" with a confidence of 0.94 and marked it as accepted, and for frames without hands it returned a confidence of 0.12 with made-up alternatives. This violated the rule of the project never to invent a prediction. The route now forwards the landmark frames to the real API and, on any failure, returns an error with no label, confidence or alternatives; the client shows that as an error. Eighteen tests cover each failure mode and check that no prediction field is present. The route was never used for any result in this report.
- **The related problems were fixed.** The routes `/api/health` and `/api/model/status` always answered healthy and loaded, and the client fell back to them when the real backend was unreachable, so the status panel could show a backend and a model that were not checked; they now proxy the real backend and answer with an error when it does not answer. The registry entry and the clip file for WATER claimed a signer review that did not happen (WATER is not an INCLUDE class and the origin of that clip is not documented); the claim was removed, a test now fails if the shipped registry claims any review, and the clip is listed as unreviewed with an undocumented origin. The registry and plan routes now count a clip only if its file exists, and an untested TypeScript copy of the text to gloss rules was removed so that there is a single implementation of the rules.
- **A clip was added to git and has been untracked.** `water.json` was committed although clip files stay out of git; it was removed from the index and is covered by `.gitignore`, but it remains in the history of that commit.

10.3 Test-split disclosure

The test split was used as follows; every use known to the author is listed.

1. The three baseline models (seed 0) were each evaluated once on the full test split (816 samples) to produce the table of Section 9.1. The earlier pilot runs used the validation split of a partial manifest only and never read test features.
2. The segmentation study of Section 9.5 evaluated streams of test samples once per rest-gap setting (three settings) after tuning on validation.
3. The browser-path benchmark of the baseline CNN on 100 test videos (Section 9.3) is a further use. It followed three end-to-end fake-camera runs on three INCLUDE test videos (Table 13), whose wrong results first exposed the picture aspect problem. The rescale was therefore designed after seeing test videos, although it was derived from the geometry and its size was not tuned on test data.
4. After the resampling routine was vectorized for speed, the baseline CNN was run over the test split once to confirm that its accuracy was unchanged.
5. Each of the three augmented models (seed 0) was evaluated once on the full test split at the end of its training run, and the six seed-0 models were evaluated once on the 312 test videos of the

larger browser corpus. The additional models of seeds 1 and 2 are evaluated in the same way once each; their count and results are **PENDING** until those runs are complete.

No augmentation range, threshold or model choice was tuned on the test split. They were chosen on validation data.

10.4 Future work

- **Record signers with identifiers on real webcams.** The recorder and the training plan are ready (signer roles S01 to S03 for training, S04 for validation and S05 for test). This is the only way to measure signer independence and the webcam gap directly, and to retrain with webcam data.
- **Review the clips and the rules with Deaf signers and ISL linguists**, and replace the rules and clips that they reject.
- **Add facial and non-manual features** to the recognizer and animate the face of the avatar.
- **Test the padding hypothesis** and the larger signer sizes with real recordings, and study cameras that are not 16:9 or 4:3.
- **Extend the vocabulary** with the 50 grammar words recorded by signers and with fingerspelling.
- **Improve continuous recognition** for signs that follow each other without a pause, with real streams instead of synthetic pauses.
- **Deploy** after the placeholder route has been removed, the avatar terms have been confirmed and the asset download has been tested.

11 Conclusion

SignBridge shows that a bidirectional ISL and English prototype can run in a web browser with the user's video staying on their own machine, with a shared gloss representation between recognition and generation. The INCLUDE dataset was processed completely and audited (4,284 samples, 262 classes), and a one-dimensional CNN trained on it reaches a top-1 accuracy of 0.9130 on the authors' predefined split, where the BiGRU and the Transformer reach 0.7929 and 0.7328. With a confidence threshold of 0.7 chosen on validation data, the live recognizer answers 0.8676 of the test samples with 0.9548 accuracy and reports the rest as unrecognized.

The most important result of the project is about measurement. A recognizer that answers 0.82 of 100 test videos correctly from stored features (0.92 top-1) answers only 0.07 of the same videos correctly (0.10 top-1), with 29 percent confident wrong answers, when it receives landmarks from a 4:3 picture whose aspect is ignored. A geometric rescale removes that failure, but a loss remains in 4:3 pictures (0.65 against 0.82 offline) that no measured factor explains, and the model is very sensitive to vertical scale and to signers who appear small in the picture. These findings became product behavior: a camera request at 16:9, the rescale, a framing guide inside the tested range and a size warning. A targeted augmentation was developed from the measurements and evaluated on three architectures; the three-seed outcome is **PENDING**.

The limits are as important as the results. No clip and no text rule has been reviewed by a signer, face features are not used, the dataset has no signer identifiers, the dataset licence is unresolved and a real webcam with real signers has not been measured. The next steps are to record signers with identifiers, to review the clips and rules with the Deaf community, and to deploy only after the problems listed in Chapter 10 have been resolved. The report, the code, the evaluation scripts and the results are arranged so that these steps can be taken without redoing the work described here.

References

[1] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison," in *Proc. IEEE Winter Conf. Applications of Computer Vision (WACV)*, 2020, pp. 1459-1469.

[2] A. Sridhar, R. G. Ganesan, P. Kumar, and M. Khapra, "INCLUDE: A Large Scale Dataset for Indian Sign Language Recognition," in *Proc. 28th ACM Int. Conf. Multimedia*, 2020, pp. 1366-1375. doi: 10.1145/3394171.3413528

[3] R. Rastgoo, K. Kiani, and S. Escalera, "Sign Language Recognition: A Deep Survey," *Expert Systems with Applications*, vol. 164, pp. 113794, 2021. doi: 10.1016/j.eswa.2020.113794

[4] S. Jiang, B. Sun, L. Wang, Y. Bai, K. Li, and Y. Fu, "Skeleton Aware Multi-modal Sign Language Recognition," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2021.

[5] P. Selvaraj, G. NC, P. Kumar, and M. Khapra, "OpenHands: Making Sign Language Recognition Accessible with Pose-based Pretrained Models across Languages," in *Proc. 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[6] A. Desai et al., "ASL Citizen: A Community-Sourced Dataset for Advancing Isolated Sign Language Recognition," in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.

[7] Google - Isolated Sign Language Recognition. Kaggle competition, https://www.kaggle.com/competitions/asl-signs, 2023.

[8] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," *arXiv preprint arXiv:1906.08172*, 2019.

[9] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device Real-time Body Pose Tracking," *arXiv preprint arXiv:2006.10204*, 2020.

[10] F. Zhang et al., "MediaPipe Hands: On-device Real-time Hand Tracking," *arXiv preprint arXiv:2006.10214*, 2020.

[11] K. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724-1734.

[12] M. Schuster and K. K. Paliwal, "Bidirectional Recurrent Neural Networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673-2681, 1997. doi: 10.1109/78.650093

[13] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30*, 2017.

[14] J. Quiñonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, Eds., *Dataset Shift in Machine Learning*. MIT Press, 2009.

[15] A. Torralba and A. A. Efros, "Unbiased Look at Dataset Bias," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2011, pp. 1521-1528. doi: 10.1109/CVPR.2011.5995347

[16] P. W. Koh et al., "WILDS: A Benchmark of in-the-Wild Distribution Shifts," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, 2021.

[17] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[18] A. D'Amour et al., "Underspecification Presents Challenges for Credibility in Modern Machine Learning," *Journal of Machine Learning Research*, vol. 23, no. 226, pp. 1-61, 2022.

[19] C. Shorten and T. M. Khoshgoftaar, "A Survey on Image Data Augmentation for Deep Learning," *Journal of Big Data*, vol. 6, no. 60, 2019. doi: 10.1186/s40537-019-0197-0

[20] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," in *Advances in Neural Information Processing Systems 30*, 2017.

[21] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017.

[22] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209-212, 1927. doi: 10.1080/01621459.1927.10502953

[23] Q. McNemar, “Note on the Sampling Error of the Difference between Correlated Proportions or Percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153-157, 1947. doi: 10.1007/BF02295996

[24] B. Efron, “Bootstrap Methods: Another Look at the Jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1-26, 1979. doi: 10.1214/aos/1176344552

[25] D. Bragg et al., “Sign Language Recognition, Generation, and Translation: An Interdisciplinary Perspective,” in *Proc. 21st Int. ACM SIGACCESS Conf. Computers and Accessibility (ASSETS)*, 2019, pp. 16-31. doi: 10.1145/3308561.3353774

Appendix A: Setup and Run Steps

All commands are Windows PowerShell commands run from the repository root. The code is portable but the provided scripts are Windows only. Node.js 20 or newer, Python 3.11 and Git are needed.

A.1 Setup

```
.\scripts\setup.ps1          # backend\.venv with PyTorch (CPU), MediaPipe, spaCy, FastAPI;
npm install in frontend
```

A.2 Files that are not in the repository

The repository does not contain the data, the trained model, the clips or the avatar, on purpose (licence and terms). Without them the application still starts and says what is missing.

Table 22. Files that are not in the repository and what happens without them.

Missing file	What happens without it	How to get it
backend/models (isl_model.pt, labels.json, threshold.json)	Sign to English says the model is not trained	train it (docs/MODEL_TRAINING.md)
frontend/public/animations/*.json	English to Sign shows no clip for every word	build the clips with the retargeting tool from the INCLUDE features
frontend/public/avatar/signer.glb	a built-in procedural figure is used	convert a Mixamo FBX that you download yourself with tools/avatar; confirm the Mixamo terms first
frontend/public/mediapipe/holistic	landmark extraction cannot load	npm run setup:mediapipe (copied from node_modules, runs before dev and build)

A.3 Run

The production build is recommended.

```
.\scripts\serve.ps1          # builds the frontend, starts the API on :8000 and the web app on
:3000
```

Open http://localhost:3000. The API needs about 15 seconds to start because it loads spaCy and the model; its interactive documentation is at http://127.0.0.1:8000/docs. To run on other ports (the web build bakes in the API address, so it must be rebuilt):

```
.\scripts\serve.ps1 -ApiPort 8110 -WebPort 3110
```

Running by hand:

```
cd frontend; npm run build; npm start
cd backend; .\.venv\Scripts\python -m uvicorn app.main:app --port 8000
```

For editing the code use `.\scripts\dev.ps1`. A Docker composition exists (run docker compose up with its build option) but was not part of the final verification.

A.4 Using the application

- **Conversation** is the default tab: sign in front of the camera, send the recognized sentence, or type a message for the avatar.
- **Sign to English**: press Start camera and allow the permission, then press Recognize sign after one isolated sign, or switch on Continuous signing and lower the hands between signs. Results below the confidence threshold are shown as unrecognized.

- **English to Sign**: type a sentence, press Translate and then Play.
- **Landmark check**: record 4 seconds, download the video and the landmark file, and compare them offline.
- **Record sign**: single sign or guided session; takes are saved as landmarks only under data/recorded.
- Keyboard shortcuts when not typing: P play or pause, A replay, U undo the last sign, C clear, S send the signed sentence, R recognize or record.

A.5 Tests

```
.\scripts\test.ps1          # pytest, tsc, eslint, vitest and the production build (stop dev
servers first)
```

A.6 Training and evaluation

```
cd backend
.\.venv\Scripts\python -m training.compare_models --manifest
..\data\processed\include_v2\manifest.csv --out ..\data\models_v2 `
    --results ..\docs\evaluation\results.md --epochs 60 --patience 10 --max-gap 3 --threads 8
# add --deploy-augment for the deployment augmentation and --checkpoint to write checkpoints
(--resume to continue a run)
```

The browser-path benchmark is described in `docs/evaluation/BROWSER_PATH.md` and its scripts are in `tools/browser_path`.

A.7 Deployment

The deployment on Vercel (web) and Hugging Face Spaces (API) is described step by step in `docs/DEPLOYMENT.md`. Nothing has been deployed. Build the Space folder with `.\scripts\make_space.ps1`, test the container locally with Docker, and follow the document for accounts, secrets and environment variables.

A.8 Rebuilding this report

```
python paper\report\make_diagrams.py     # architecture and pipeline diagrams (matplotlib)
python paper\report\make_charts.py       # charts from docs/evaluation and the skeleton figure
python paper\report\build_report.py      # the Word file; Word is used to update the table of
contents and to count pages
```

When the three-seed runs are complete, a summary file `docs/evaluation/three_seed_summary.json` with the keys used in the three-seed table fills the pending cells at the next build.

Appendix B: API Reference

The interactive documentation is served by the backend at /docs. Every response carries an X-Process-Time-Ms header, and requests with a body above 2 MB are refused with status 413.

Table 23. REST and WebSocket endpoints.

Endpoint	Purpose
GET /api/health	status, service name and version
GET /api/model/status	state of the recognition model: not trained, loaded or error
GET /api/avatar/animations	the valid clip files and any invalid ones
GET /api/avatar/registry	per-sign provenance and computed status (no clip, clip unreviewed, verified, missing asset) and totals
POST /api/nlp/text-to-gloss	text of 1 to 500 characters to gloss, tokens with fingerspelling letters, non-manual markers, tense and dropped words with reasons
POST /api/nlp/gloss-to-text	gloss list to English by heuristic rules
POST /api/translation/english-to-gloss	English to gloss with per-word status, unsupported words and coverage
POST /api/translation/gloss-to-english	gloss list (1 to 50) to English with unsupported tokens
GET /api/translation/vocabulary	glosses, English meanings and whether each has a clip
POST /api/avatar/plan	per-token availability and clip address for a gloss list
POST /api/landmarks/extract	validate and normalize 1 to 300 landmark frames and report usability and warnings
POST /api/signs/recognize	predictions for one isolated sign from landmark frames; 503 without a model, 422 for unusable landmarks
POST /api/signs/translate	recognition, confidence threshold, label to gloss and English for one isolated sign
WS /ws/predict	live recognition: isolated and continuous modes (messages in Chapter 5)
POST /api/record/sign	store a landmark take after a quality check, optionally with a clip; 403 when recording is disabled
GET /api/record/plan	the glosses of a guided session, priority words first
GET /api/record/takes	saved takes per gloss for one signer, so that a session can resume

Table 24. Configuration through environment variables (prefix SIGNBRIDGE_).

Variable	Meaning
CORS_ORIGINS	JSON list of allowed web origins
CORS_ORIGIN_REGEX	one regular expression for further origins, for example preview deployments
WS_CHECK_ORIGIN	refuse WebSocket connections from origins that CORS would refuse
RECORDING_ENABLED	turn the recorder endpoints on or off
MODEL_DIR, ANIMATION_DIR	where the model and the clips are read
MIN_CONFIDENCE	default confidence threshold when the model has none
MAX_BODY_BYTES	request body limit